University of Southern Denmark

Department of Mathematics and Computer Science

SPD801: Master Thesis in Computer Science

Formalising πLL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

28th May 2026

Abstract

Words words words words words words words words words

Ord ord ord ord ord ord ord ord ord ord ord

Contents

1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π-calculus.

This line of work has been further developed by Montesi and Peressotti [2021], who provide a unified account reconciling linear logic, the π-calculus, and their metatheory. Their framework serves as the formal basis for the system πLL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, a small mistake in the application of a typing rule can arise when the intuitive order of names does not match the formal order imposed by the rule. It is tempting to assign types following the order in which names appear syntactically, but a rule may assign types according to the role each name plays instead. Thus, despite the rule itself being correct, its application is easy to get wrong because the intuitive and formal interpretations pull in opposite directions. Such mistakes does not reflect a flaw in the underlying theory, but they are easy to overlook in paper presentations where rule application is guided by intuition rather than by strict syntactic structure. This makes the system a natural candidate for mechanisation, since in proof assistants such as Lean, every rule application must instantiate the required variables and side conditions explicitly.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanization. Mechanization provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanization in this work is to validate and internalize existing results within a proof assistant. This allows us to eliminate ambiguity arising from paper presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanized developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanized formalization of the π-calculus-based linear logic system πLL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalisation of the syntax and typing system of πLL in Lean, including processes, session types, environments, hyperenvironments, typing judgements, and substitution operations for both names and types.
- A binding architecture based on De Bruijn indices for type variables and a locally nameless representation for process binders, avoiding explicit reasoning up to α-equivalence.
- A formalisation of the operational semantics for the multiplicative fragment of πLL.

- A mechanised development of the session-fidelity proof architecture for the multiplicative fragment, with the remaining proof obligation isolated to the `res` case.

Scope of the formalization. This thesis focuses on a mechanized reconstruction of the πLL system of [Montesi and Peressotti 2021], with particular emphasis on its multiplicative fragment. The syntactic and typing components of πLL have been fully mechanized, including processes, session types, typing judgments, and substitution operations. The operational semantics are defined for the multiplicative fragment, while the corresponding session-fidelity proof currently leaves the `res` case as the remaining proof obligation.

Consequently, metatheoretic results concerning typing apply to the full system, while results involving operational semantics are restricted to the multiplicative fragment. The aim of this work is not to extend the theory, but to provide a mechanised account of an existing theoretical framework and to identify the remaining proof obligations needed for its full verification.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of [Montesi and Peressotti 2021], We cover the full πLL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment πMLL when discussing operational semantics.

Following [Montesi and Peressotti 2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \parr B$, together with the corresponding units 1 and $\bot$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \parr B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B, whereas $A \parr B$ corresponds to receiving a value of type A and proceeding as B. The units 1 and $\bot$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1}\,1 \qquad \frac{\vdash \Gamma}{\vdash \Gamma, \bot}\,\bot \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B}\,\otimes \qquad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \parr B}\,\parr$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B, each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B.

The unit rule rule 1 introduces the unit 1, which represents the absence of further obligations. It can be understood as an "empty output", corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\bot$ and $⅋$ describe the behavior of the connective $A ⅋ B$ and its unit $\bot$. The $⅋$ rule combines resources within a single context, reflecting a form of interaction where both components are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B. The unit $\bot$ represents the absence of further input, acting as an "empty input" and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the rule 1 and combined via rule $\otimes$ to derive $1 \otimes 1$:

$$\dfrac{\dfrac{}{\vdash 1}\,1 \qquad \dfrac{}{\vdash 1}\,1}{\vdash 1 \otimes 1}\,\otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying πLL.

2.2 The π-Calculus: Processes and Communication

The π-calculus [Milner et al. 1992] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of Montesi and Peressotti [2021], which adopts Wadler's convention of using square brackets '[-]' for output and round parentheses '(-)' for input.

Programs in πMLL are processes (P, Q, R, ...), which communicate over names (x, y, z, ...) representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in 2.3. We assume an infinite set of names with decidable equality. The process terms of πMLL are defined by the following grammar:

$P, Q ::=$	$x[y].P$	*output y on x and continue as P*
$\mid$	$x(y).P$	*input y on x and continue as P*
$\mid$	$x[].P$	*output (empty message) on x and continue as P*
$\mid$	$x().P$	*input (empty message) on x and continue as P*
$\mid$	$\nu xy\, P$	*name restriction (cut)*
$\mid$	$P \mid Q$	*parallel composition*
$\mid$	$x[\textsc{l}].P$	*select left on x and continue as P*
$\mid$	$x[\textsc{r}].P$	*select right on x and continue as P*
$\mid$	$x.\mathsf{case}\{\textsc{l}{:}P, \textsc{r}{:}Q\}$	*offer a binary choice between P or Q on x*
$\mid$	$x[A].P$	*output type A on x and continue as P*
$\mid$	$x(X).P$	*input a type as X on x and continue as P*
$\mid$	$!x\langle z_1, \ldots, z_n\rangle.\{P\}$	*server (replicable process) on x depending on servers on $z_1 \ldots z_n$*
$\mid$	$x[\textsc{use}].P$	*consume a server on x and continue as P*
$\mid$	$x[\textsc{dup}](x').P$	*duplicate server x as x' in P*
$\mid$	$x[\textsc{disp}].P$	*dispose of a server on x*

$$
\begin{array}{llll}
 & \mid & x{\leftrightarrow}y & \textit{link } x \textit{ and } y \\
 & \mid & \mathbf{0} & \textit{terminated process}
\end{array}
$$

Term $x[y].P$ allocates a fresh name y, outputs it over x, and proceeds as P. Dually, $x(y).P$ inputs a name over x, binding it to y in P. Terms $x[].P$ and $x().P$ model empty output and input. Restriction $\boldsymbol{\nu} xy\, P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\boldsymbol{\nu} xy\, P = \boldsymbol{\nu} yx\, P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and $\mathbf{0}$ is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \boldsymbol{\nu} xz\, (x[].\mathbf{0} \mid z().y[].\mathbf{0})$. This process creates a private session between the restricted endpoints x and z. The left process, $x[].\mathbf{0}$, signals termination on x before terminating, while the right process, $z().y[].\mathbf{0}$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In πLL, session types are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types as presented in [Montesi and Peressotti 2021] following [Carbone et al. 2016; Wadler 2014] is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B, while its dual $A \parr B$ describes receiving. The units 1 and $\perp$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \,\&\, B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined as follows:

$$
\begin{array}{llllll}
A, B ::= & A \otimes B & \textit{send } A\textit{, proceed as } B & \mid & A \parr B & \textit{receive } A\textit{, proceed as } B \\
\mid & 1 & \textit{empty output, unit for } \otimes & \mid & \perp & \textit{empty input, unit for } \parr \\
\mid & A \,\&\, B & \textit{offer } A \textit{ or } B & \mid & A \oplus B & \textit{select } A \textit{ or } B \\
\mid & \exists X.A & \textit{existential, type output} & \mid & \forall X.A & \textit{universal, type input} \\
\mid & ?A & \textit{client request} & \mid & !A & \textit{server accept} \\
\mid & X & \textit{type variable} & \mid & {}^{\perp}X & \textit{dual of type variable}
\end{array}
$$

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, i.e. $(A^{\perp})^{\perp} = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$
(A \otimes B)^{\perp} = A^{\perp} \parr B^{\perp} \qquad 1^{\perp} = \perp \qquad (A \oplus B)^{\perp} = A^{\perp} \,\&\, B^{\perp}
$$
$$
(!A)^{\perp} = ?A^{\perp} \qquad (\exists X.A)^{\perp} = \forall X.A^{\perp} \qquad (X)^{\perp} = X^{\perp}
$$

2.4 Environments, Hyperenvironments and Judgements

Typing in πLL is organised into two layers. Environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in [Montesi and Peressotti 2021], *environments* $(\Gamma, \Delta, \Xi, \dots)$ are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \dots, x_n : A_n$, where each x_i is associated to its respective A_i, for $1 \le i \le n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then Γ, Δ is the environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using "," as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \qquad \Gamma, \Delta = \Delta, \Gamma \qquad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by handled by *hyperenvironments* $(\mathcal{G}, \mathcal{H}, \mathcal{I}, \dots)$, which are collections of environments joined using the parallel operator '$\mid$'. They can be written as follows $\mathcal{G} = \Gamma_1, \dots, \Gamma_n$, where $\Gamma_1, \dots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} \mid \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using '$\mid$' the sum and $\varnothing$ as unit, where $\varnothing$ is the hyperenvironment containing no environments:

$$\mathcal{G} \mid \varnothing = \mathcal{G} \qquad \mathcal{G} \mid \mathcal{H} = \mathcal{H} \mid \mathcal{G} \qquad (\mathcal{G} \mid \mathcal{H}) \mid \mathcal{I} = \mathcal{G} \mid (\mathcal{H} \mid \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations $(\mathcal{D}, \mathcal{E}, \mathcal{F}, \dots)$ are written as $\overset{\mathcal{D}}{\vdash P :: \mathcal{G}}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P, appearing in the judgement concluding a derivation is well-typed.

Multiplicative Fragment

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \textsc{cut} \qquad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \textsc{mix} \qquad \frac{}{\vdash \mathbf{0} :: \varnothing}\ \textsc{mix}_0$$

$$\frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B}\ \otimes \qquad \frac{\vdash P :: \varnothing}{\vdash x[].P :: x : 1}\ 1 \qquad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \parr B}\ \parr \qquad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \perp}\ \perp$$

Additional rules for Additives, Quantifiers and Exponentials

$$\frac{\vdash P :: \Gamma, x : A}{\vdash x[\textsc{l}].P :: \Gamma, x : A \oplus B}\ \oplus_1 \qquad \frac{\vdash P :: \Gamma, x : B}{\vdash x[\textsc{r}].P :: \Gamma, x : A \oplus B}\ \oplus_2 \qquad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\mathrm{case}\{\textsc{l}{:}P, \textsc{r}{:}Q\} :: \Gamma, x : A \,\&\, B}\ \&$$

$$\frac{}{\vdash x \leftrightarrow y :: x : A^{\perp}, y : A}\ \textsc{ax} \qquad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\textsc{use}].P :: \Gamma, x : ?A}\ ? \qquad \frac{\vdash P :: \Gamma}{\vdash x[\textsc{disp}].P :: \Gamma, x : ?A}\ \textsc{w} \qquad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\textsc{dup}](x').P :: \Gamma, x : ?A}\ \textsc{c}$$

$$\frac{\vdash P :: z_1 : ?B_1, \dots, z_n : ?B_n, x : A}{\vdash !x\langle z_1, \dots, z_n \rangle.\{P\} :: z_1 : ?B_1, \dots, z_n : ?B_n, x : !A}\ ! \qquad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B}\ \exists \qquad \frac{\vdash P :: \Gamma, x : B \quad X \notin \mathrm{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B}\ \forall$$

Fig. 1. πLL, typing rules.

The multiplicative rules form the core of the system. Rule $\textsc{cut}$ composes two processes in P sharing dual endpoints $x : A$ and $y : A^{\perp}$, hiding them via restriction. This is the logical equivalent of communication. Rule $\textsc{mix}$ and rule $\textsc{mix}_0$ compose independent processes in parallel and introduce the terminated process, respectively.

Rule $\otimes$ and rule $\parr$ type output and input. Note that rule $\otimes$ types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, rule $\parr$ keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The units rule 1 and rule $\bot$ type empty output and empty input, where rule 1 requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to $\mathbf{0}$.

The additive rules (rule $\oplus_1$, rule $\oplus_2$, and rule $\&$) type selection and offering of alternative processing paths. Note that rule $\&$ requires both branches to be typed in the *same* context Γ, reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The rule $!$ requires all free names except x to be typed using $?$, enforcing that a server only depends on other servers. The rule $?$, rule $\textsc{w}$, and rule $\textsc{c}$ allow a client to use, discard, or duplicate a server respectively.

The rule $\textsc{ax}$ types a link between two dual endpoints, $x{\leftrightarrow}y$, forwarding all messages sent on x safely to y and vice versa. Rule $\exists$ and rule $\forall$ type polymorphic communication, where the side condition $X \notin \mathrm{ft}(\Gamma)$ in rule $\forall$, ensures the introduced type variable, X, does not accendentally shadow one already existing in Γ.

Example 2.2. The process from Example 2.1 has the typing $\vdash \nu zx\,(z().y[].\mathbf{0} \mid x[].\mathbf{0}) :: y : 1$, as shown by the derivation below:

$$
\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\textsc{mix}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\textsc{mix}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}\,\textsc{mix}}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\textsc{cut}
$$

Note that, for the typing context $x : 1 \mid y : 1, z : \bot$ to align with the premise of rule $\textsc{cut}$, we utilise the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \varnothing$ and $\Gamma = \bullet$. Under these instantiations, the premise of rule $\textsc{cut}$: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}$ can be viewed as $x : A \mid \Delta, y : A^{\perp}$. This effectively instantiates the $\textsc{cut}$ over the dual types $A = 1$ and $A^{\perp} = \bot$ on channels x and y, with $\Gamma = y : 1$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for a creating a valid $\textsc{cut}$ (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in Figure 2. The full LTS is given in Montesi and Peressotti [2021].

Remark 2.3. The operational semantics rely explicitly on the structural equivalence (rule $=_{\alpha}$) to rename restricted variables via α-equivalence prior to a transition. In paper-based presentations, α-equivalence is often handled implicitly through Barendregt's Variable Convention. In a mechanised setting, however, reasoning over named variables requires freshness conditions, capture-avoiding

$$\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \varnothing\end{array}}{\vdash x[].P :: x:1}\,1} \xrightarrow{x[]} \boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \varnothing\end{array}} \qquad \boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A \mid \Delta, x:B\end{array}}{\vdash x[x'].P :: \Gamma, \Delta, x:A \otimes B}\,\otimes} \xrightarrow{x[x']} \boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A \mid \Delta, x:B\end{array}}$$

$$\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma\end{array}}{\vdash x().P :: \Gamma, x:\bot}\,\bot} \xrightarrow{x()} \boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma\end{array}} \qquad \boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A, x:B\end{array}}{\vdash x(x').P :: \Gamma, x:A \parr B}\,\parr} \xrightarrow{x(x')} \boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A, x:B\end{array}}$$

$$\dfrac{\boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array}} \xrightarrow{l} \boxed{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array}} \quad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\,\text{MIX}} \xrightarrow{l} \boxed{\dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P' \mid Q :: \mathcal{G}' \mid \mathcal{H}}\,\text{MIX}}}\,\text{PAR}_1$$

$$\dfrac{\boxed{\begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}} \xrightarrow{l} \boxed{\begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array}} \quad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\,\text{MIX}} \xrightarrow{l} \boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array}}{\vdash P \mid Q' :: \mathcal{G} \mid \mathcal{H}'}\,\text{MIX}}}\,\text{PAR}_2$$

$$\dfrac{\boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array}} \xrightarrow{l} \boxed{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array}} \quad \boxed{\begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}} \xrightarrow{l'} \boxed{\begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array}} \quad \mathrm{i}(l \mid l') \cap \mathrm{f}(P \mid Q) = \emptyset}{\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\,\text{MIX}} \xrightarrow{l \mid l'} \boxed{\dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array} \quad \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array}}{\vdash P' \mid Q' :: \mathcal{G}' \mid \mathcal{H}'}\,\text{MIX}}}\,\text{SYN}$$

$$\dfrac{P =_\alpha Q \quad \boxed{\begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{G}\end{array}} \xrightarrow{l} \boxed{\begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{G}'\end{array}}}{\boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array}} \xrightarrow{l} \boxed{\begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{G}'\end{array}}}\,=_\alpha \qquad \dfrac{\boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot\end{array}} \xrightarrow{x[] \mid y()} \boxed{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G} \mid \Gamma\end{array}}}{\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot\end{array}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma}\,\text{CUT}} \xrightarrow{\tau} \boxed{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G} \mid \Gamma\end{array}}}\,1\bot$$

$$\dfrac{\boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp\end{array}} \xrightarrow{x[x'] \mid y(y')} \boxed{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp\end{array}}}{\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp\end{array}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\,\text{CUT}} \xrightarrow{\tau} \boxed{\dfrac{\dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp\end{array}}{\vdash \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, \Xi, y:B^\perp}\,\text{CUT}}{\vdash \nu xy\, \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\,\text{CUT}}}\,\otimes\parr$$

$$\dfrac{\boxed{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp\end{array}} \xrightarrow{l} \boxed{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp\end{array}} \quad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\boxed{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp\end{array}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\,\text{CUT}} \xrightarrow{l} \boxed{\dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp\end{array}}{\vdash \nu xy\, P' :: \mathcal{G}' \mid \Gamma', \Delta'}\,\text{CUT}}}\,\text{RES}$$

Fig. 2. πMLL, transition rules for derivations.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}\,\text{MIX}}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\text{CUT}$$

$\downarrow \tau$ (by rules SYN and $1\bot$, and the transition axioms for rules 1 and $\bot$)

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0 \qquad \dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash \mathbf{0} \mid y[].\mathbf{0} :: \varnothing \mid y : 1}\,\text{MIX}$$

$\downarrow y[]$ (by rule PAR_2, the transition axiom for rule 1)

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0 \qquad \dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash \mathbf{0} \mid \mathbf{0} :: \varnothing \mid \varnothing}\,\text{MIX}$$

$\downarrow \equiv$ (by process congruence, and the monoidal structure of typing environments)

$$\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0$$

Fig. 3. An execution for the derivation in Example 2.2.

substitution, and explicit renaming lemmas, introducing substantial proof overhead [Aydemir et al. 2005; Charguéraud 2012]. As discussed in Section 3, this motivates the use of a different binding representation for the syntax of πLL, one designed to reduce explicit α-equivalence reasoning and simplify the treatment of bound variables.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l|l') \mid l, l' \in Act, \mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

The free and introduced names of labels are defined by $\mathrm{f}(x[]) = \mathrm{f}(x()) = \mathrm{f}(x[y]) = \mathrm{f}(x(y)) = \{x\}$, $\mathrm{f}(\tau) = \mathrm{i}(\tau) = \emptyset$, and $\mathrm{f}(l \mid l') = \mathrm{f}(l) \cup \mathrm{f}(l')$; and introduced names by $\mathrm{i}(x[]) = \mathrm{i}(x()) = \emptyset$, $\mathrm{i}(x[y]) = \mathrm{i}(x(y)) = \{y\}$, and $\mathrm{i}(l \mid l') = \mathrm{i}(l) \cup \mathrm{i}(l')$. The parallel label $l \mid l'$ additionally requires $\mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset$, ensuring no two components introduce the same name.

Example 2.4. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (1, $\bot$, $\otimes$, $⅋$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l|l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\bot$ and $\otimes⅋$ simplify the term into an internal τ-transition, modeling the synchronisation of the session.

2.6 Process Semantics and Erasure

While the typing derivations of πLL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π-calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti 2021] and define an erasure function, $proc : Der \longrightarrow Proc$, which strips away the typing information from a derivation. Recursively, *proc* maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed πLL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$
\begin{prooftree}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash v[].\mathbf{0} :: v : 1$}
\RightLabel{$\bot$}
\UnaryInfC{$\vdash w().v[].\mathbf{0} :: v : 1, w : \bot$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash x[].\mathbf{0} :: x : 1$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash y[].\mathbf{0} :: y : 1$}
\RightLabel{$\bot$}
\UnaryInfC{$\vdash z().y[].\mathbf{0} :: y : 1, z : \bot$}
\RightLabel{MIX}
\BinaryInfC{$\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot$}
\RightLabel{MIX}
\BinaryInfC{$\vdash (w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v : 1, w : \bot \mid x : 1 \mid y : 1, z : \bot$}
\RightLabel{CUT}
\UnaryInfC{$\vdash \boldsymbol{\nu} xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v : 1, w : \bot \mid y : 1$}
\end{prooftree}
$$

Fig. 4. A derivation for the process $\boldsymbol{\nu} xz\, w().y[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}$

Example 2.5. Consider the well-typed process P formed by adding a third parallel component to the derivation of Example 2.2 before the cut (A derivation of which can be seen on Figure 4):

$$P = \boldsymbol{\nu} xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$$

The component containing w and v is independent from the component containing x, z, and y. This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$
\begin{array}{ccc}
P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0} & P \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{w()} \xrightarrow{v[]} \mathbf{0} \\
P \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{\tau} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0}
\end{array}
$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated in [Montesi and Peressotti 2021], this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times Lbl$ be endofunctors representing the states and labeled transitions, respectively. The erasure function *proc* acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields an Erasure theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

Theorem 2.6 (Erasure). *The process LTS (lts_p) enjoys erasure with respect to the derivation LTS (lts_d). Concretely, for any valid typing derivation $\mathcal{D}$ and label $l \in Lbl$:*

1. ***Soundness of Erasure:*** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*
2. ***Completeness of Erasure:*** *If $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.*

$$x[x'].P \xrightarrow{x[x']} P \qquad x(x').P \xrightarrow{x(x')} P \qquad x[].P \xrightarrow{x[]} P \qquad x().P \xrightarrow{x()} P$$

$$\frac{P \xrightarrow{l} P' \quad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q}\ \textsc{par}_1 \qquad \frac{Q \xrightarrow{l} Q' \quad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'}\ \textsc{par}_2$$

$$\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l|l'} P' \mid Q'}\ \textsc{syn} \qquad \frac{P \xrightarrow{x[x']|y(y')} P'}{\nu xy\, P \xrightarrow{\tau} \nu xy\, \nu x'y'\, P'}\ \otimes\!\invamp$$

$$\frac{P \xrightarrow{x[]|y()} P'}{\nu xy\, P \xrightarrow{\tau} P'}\ 1\bot \qquad \frac{P \xrightarrow{l} P' \quad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\nu xy\, P \xrightarrow{l} \nu xy\, P'}\ \textsc{res} \qquad \frac{P =_\alpha Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'}\ =_\alpha$$

$$\begin{aligned}
\mathrm{i}(l|l') &= \mathrm{i}(l) \cup \mathrm{i}(l') \\
\mathrm{f}(l|l') &= \mathrm{f}(l) \cup \mathrm{f}(l') \\
\mathrm{i}(\tau) &= \emptyset \\
\mathrm{f}(\tau) &= \emptyset \\
\mathrm{i}(x[]) = \mathrm{i}(x()) &= \emptyset \\
\mathrm{f}(x[]) = \mathrm{f}(x()) &= \{x\} \\
\mathrm{i}(x[x']) = \mathrm{i}(x(x')) &= \{x'\} \\
\mathrm{f}(x[x']) = \mathrm{f}(x(x')) &= \{x\}
\end{aligned}$$

Fig. 5. πMLL, transition rules for processes.

Corollary 2.7 (Typability Preservation). *If P is well-typed and $P \xrightarrow{l} P'$, then P' is well-typed.*

This theorem allows us to define the operational target for πLL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

Remark 2.8. The untyped LTS for processes relies heavily on explicit side conditions regarding free and introduced names to prevent the collision of bound variables (e.g., ensuring $i(l) \cap i(l') = \emptyset$). However, when operating at the level of typing derivations, these side conditions become inherently redundant. Because πLL enforces linear resource management, the structural rules of the typed LTS dictate that hyperenvironments can only be merged when their domains are strictly disjoint. Consequently, any well-typed derivation inherently satisfies the name-distinctness and freshness requirements of the underlying untyped calculus, shifting the burden of safety from dynamic transition checks to static structural typing.

Proposition 2.9. *The operational semantics of πLL processes strictly respects the introduction of names via transition labels. Formally, if $P \xrightarrow{l} P'$, then $\mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset$, and If P is well-typed, then $\mathrm{i}(l) = \mathrm{f}(P') \setminus \mathrm{f}(P)$ (TODO: Evaluate relevance - delete depending on if this property is mentioend in later proofs).*

Remark 2.10. Proposition 2.9 is vital for mechanising the metatheory of πLL. Because proof assistants tools require explicit tracking of fresh and bound variable to prevent accidental capture, this proposition acts as the formal bridge between the dynamic semantics and the static typing. It guarantees that a well-typed process evolves predictably, ensuring that it does not spontaneously generate resources during a transition. (TODO: same as Proposition 2.9)

2.7 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In πLL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow Montesi and Peressotti [2021] and define a function $env : Der \longrightarrow Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, Montesi and Peressotti [2021] derive a LTS specifically for typing environments (lts_e), shown in Figure 6.

$$x:1 \xrightarrow{x[]} \varnothing \qquad \Gamma, x:\bot \xrightarrow{x()} \Gamma \qquad \Gamma,\Delta, x:A\otimes B \xrightarrow{x[x']} \Gamma, x':A \mid \Delta, x:B \qquad \Gamma, x:A \parr B \xrightarrow{x(x')} \Gamma, x':A, x:B$$

$$\frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G}' \mid \mathcal{H}}\ \text{PAR}_1 \qquad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G} \mid \mathcal{H}'}\ \text{PAR}_2 \qquad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l \| l'} \mathcal{G}' \mid \mathcal{H}'}\ \text{SYN}$$

$$\frac{\mathcal{G} \mid \Gamma, x:A^\perp \mid \Delta, y:A \xrightarrow{l} \mathcal{G}' \mid \Gamma', x:A^\perp \mid \Delta', y:A}{\mathcal{G} \mid \Gamma, \Delta \xrightarrow{l} \mathcal{G}' \mid \Gamma', \Delta'}\ \text{RES} \qquad \frac{\mathcal{G} \mid x:1 \mid \Gamma, y:\bot \xrightarrow{x[] \| y()} \mathcal{G} \mid \Gamma}{\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma}\ 1\bot$$

$$\frac{\mathcal{G} \mid \Gamma, \Delta, x:A\otimes B \mid \Xi, y:A^\perp \parr B^\perp \xrightarrow{x[x'] \| y(y')} \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\mathcal{G} \mid \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \otimes\parr$$

Fig. 6. πMLL, transition rules for typing environments.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x:\bot$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalised by the τ-reflexivity axiom, $\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ-action.

The requirement that a process can only perform actions permitted by its typing environment is known as **Session Fidelity**. Just as the *proc* function established operational correspondence for processes, the *env* function establishes a correspondence for environments.

While the rules in Figure 6 define how environments can transition in isolation, the execution of a well-typed program requires the process and its environment to step in unison. This co-dependent relationship has processes only perform actions permitted by its typing environment, while the environment dynamically mirrors the physical control flow of the process, and is formally captured by the property of **Session Fidelity**. Just as the *proc* function established operational correspondence for the untyped calculus, the *env* function establishes this exact behavioral correspondence for the typing contexts.

Theorem 2.11 (Session Fidelity). *Let $\mathcal{D}$ be a valid typing derivation such that $env(\mathcal{D}) = \mathcal{G}$ and $proc(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition*
$\mathcal{G} \xrightarrow{l} \mathcal{G}'$,
and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.

To illustrate Session Fidelity in action, we can observe how the environment is constrained to follow the exact execution traces of the process it types.

Example 2.12 (Session Fidelity in Execution). Recall the process derivation from Example 2.5, typed by the environment $\mathcal{G} = v:1, w:\bot \mid y:1$. The process begins by performing an observable input action on w. By Session Fidelity, the environment must faithfully mimic this action, consuming the corresponding type:

$$v:1, w:\bot \mid y:1 \xrightarrow{w()} v:1 \mid y:1$$

The resulting derivative of the process is now typed by this new environment. At this intermediate stage, if we were to view the environment LTS in isolation, the pure semantics would theoretically allow an output on y to fire immediately, as $y:1$ is an available, unblocked resource.

However, Session Fidelity dictates a strict co-dependence. The environment merely mirrors the process, and the underlying process term strictly guards the $y[]$ action behind a prefix, requiring a prior τ-synchronisation. Therefore, the environment must wait. When the process finally resolves its internal communication, the environment mimics it via the τ-reflexivity axiom, remaining structurally unchanged:

$$v : 1 \mid y : 1 \xrightarrow{\tau} v : 1 \mid y : 1$$

Only after this synchronisation unblocks the continuation can the process perform the $y[]$ action, which the environment once again mirrors by consuming the final resource:

$$v : 1 \mid y : 1 \xrightarrow{y[]} v : 1$$

Remark 2.13 (The Mechanisation Gap). Session Fidelity is the ultimate touchstone for validating the safety of a session-typed language. However, verifying this property in a proof assistant exposes a stark gap between paper mathematics and mechanised logic. On paper, the invariant that an environment *dynamically mirrors* a process is highly intuitive, and transitions are easily massaged using structural equivalences and implicit variable conventions. In a machine, however, formally proving that types are consumed correctly requires a rigorous, algorithmic representation of bound variables and explicit freshness. The theoretical elegance of the environment LTS cannot be mechanised without first establishing a concrete architectural foundation for these variables, which dictates the design choices discussed in the following chapters.

2.8 Managing Bound Variables

Having established the operational semantics of πLL and the ultimate goal of verifying Session Fidelity, we must address the treatment of bound variables and freshness, a central technical challenge that underpins the entire formalisation.

As we have seen, many constructs in πLL introduce scoped identifiers. Restriction binds pairs of session endpoints, input constructs bind received channel names, polymorphic communication binds type variables, and duplication constructs introduce fresh channel endpoints. These bindings appear throughout both the process syntax and the labelled transition system.

The concrete choice of bound and introduced names is semantically irrelevant provided the binding structure is preserved. For example, renaming a restricted channel does not change the behaviour of a process. However, reasoning formally about substitutions, transitions, and typing derivations requires these names, scopes, and freshness conditions to be tracked explicitly.

This principle is formalised as α-*equivalence*, which identifies terms that differ only in the names of their bound variables. For instance, the following two processes are α-equivalent:

$$\boldsymbol{\nu} ab\,(a[].\mathbf{0} \mid b().x[].\mathbf{0}) =_\alpha \boldsymbol{\nu} zw\,(z[].\mathbf{0} \mid w().x[].\mathbf{0})$$

The operational semantics of πLL relies on α-equivalence using rule $=_\alpha$ to rename bound variables on the fly, ensuring that restricted scopes and input prefixes do not accidentally capture free names as processes evolve.

Substitution raises this exact issue. Consider the substitution where the free name x is replaced by z in the right-hand process from the previous example:

$$\boldsymbol{\nu} zw\,(z[].\mathbf{0} \mid w().x[].\mathbf{0})$$

Here z is already bound by the restriction, so naively replacing x with z would cause the restriction to capture the newly inserted z. Before the substitution can proceed, and because restriction binds session endpoints as pairs, both the bound z and w must be α-renamed to fresh names, such as to a and b. Similarly, any binder within the process that would capture the substituted name must also be renamed before substitution continues.

In paper-based presentations this is commonly handled by *Barendregt's variable convention*, which assumes bound variables are always chosen distinct from free variables and one another whenever convenient. This makes proofs readable but remains an informal meta-level assumption.

A proof assistant requires every freshness condition and renaming step to be represented and verified explicitly [Aydemir et al. 2005]. A direct mechanisation using named variables therefore introduces substantial proof overhead for capture avoidance and α-equivalence reasoning. To avoid this, the formalisation adopts a *locally nameless* representation of binding, which eliminates α-equivalence as a proof obligation by construction. This is the subject of the following chapter.

3 Binding Architecture

This chapter develops the locally nameless binding architecture used throughout the formalisation. We begin by stating the design goals that motivate the approach (Section 3.1), then introduce the locally nameless representation and its core operations (Sections 3.2 and 3.3), and conclude with co-finite quantification (Section 3.4), which governs how inductive definitions interact with binders.

3.1 Design Goals and the Approach

This chapter introduces the binding representation used throughout the formalisation. The representation is motivated by three primary design goals:

(1) **Eliminate α-equivalence overhead.** α-equivalent terms should be represented identically, reducing α-equivalence reasoning to definitional equality within the proof assistant.
(2) **Preserve human readability.** Terms should remain explicit and concrete, keeping substitution and freshness reasoning close to their paper-based presentation.
(3) **Avoid informal meta-level conventions.** The representation should not rely on Barendregt's convention or unverified freshness assumptions; capture avoidance and freshness conditions should instead be enforced by construction.

Although both channels and type variables involve binding, they exhibit fundamentally different operational behaviour. Session channels participate in dynamic communication and substitution across concurrent processes, while polymorphic type variables remain statically scoped within typing derivations. The formalisation therefore adopts different binding strategies for these two classes of identifiers while maintaining a common underlying binding architecture.

To achieve these goals, we adopt a locally nameless binding architecture for the calculus [Charguéraud 2012], separating free identifiers from bound occurrences. Bound variables are represented using indices, causing α-equivalent terms to coincide structurally, while free variables remain explicit and readable.

This separation simplifies substitution by restricting it to free variables while leaving bound indices untouched, thereby avoiding accidental capture by construction. As a result, many proofs that traditionally require substantial renaming infrastructure reduce instead to structural recursion and index manipulations.

The remainder of this chapter develops the theoretical machinery underlying this representation.

3.2 The Locally Nameless Representation

The locally nameless representation, as presented by Charguéraud [2012], separates variables into two disjoint syntactic categories: free names and bound indices. This combines the readability of named syntax for free variables with a structural representation of binding.

(1) **Bound variables** are represented using *De Bruijn indices* [de Bruijn 1972], where each index denotes the distance to its corresponding binder. Since the representation is purely positional, α-equivalent terms become structurally identical.

(2) **Free variables** retain explicit names, preserving the readability of open terms and allowing standard name-based reasoning about environments and transition labels.

By structurally separating these two concepts, the locally nameless approach simplifies standard operations such as capture-avoiding substitution. Since free variables and bound variables inhabit distinct syntactic categories, substitution on free variables is defined to not interact with bound indices. We apply this theory to both the structural and operational layers of πLL.

LN Syntax for Types. In the original presentation of πLL, polymorphic types rely on standard named variables. Under the locally nameless architecture, we expand the syntax of type variables to explicitly distinguish between free and bound occurrences. The grammar for types is updated as follows:

$$
\begin{array}{rcll}
T & ::= & \mathrm{free}(x) \mid \mathrm{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\
A, B & ::= & \cdots \mid \exists.A \mid \forall.A \mid X \mid X^{\perp} & (\text{where } X \in T)
\end{array}
$$

Here, a free type variable carries a unique identifier x, while a bound type variable carries a De Bruijn index k. For example, the polymorphic type $\forall X.\exists Y.(X \parr Y)$ is structurally represented without names using the new syntax as $\forall.\exists.(\mathrm{bound}(1) \parr \mathrm{bound}(0))$, where 0 points to the inner existential binder and 1 points to the outer universal binder.

LN Syntax for Processes. Similarly, the standard process syntax displayed in Section 2.2 is updated to reflect the locally nameless separation of channel endpoints. A channel is split into its bound and free counterparts, where bound channel names are represented by indices, while free channels remain explicit identifiers.

Naively removing the bound names from the process syntax would turn binding operations such as $x[y].P$ and $x(y).P$ into $x[].P$ and $x().P$ making them syntactically indistinguishable from their non-binding unit counterparts. To combat this we annotate anonymous binders using placeholder symbols. We use $\bullet$ for a bound process name, and $\diamond$ for a bound type variable.

$$
\begin{array}{rcll}
\mathit{Channel} & ::= & \mathrm{free}(x) \mid \mathrm{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\
P, Q & ::= & \cdots \mid x[\bullet].P \mid x(\bullet).P \mid \nu(\bullet,\bullet).\,P \mid x(\diamond).P & (\text{where } x \in \mathit{Channel})
\end{array}
$$

While the visual anonymity of these constructors is evident from the grammar, their mechanical implementation relies on precise index management. For a multi-name binder like restriction, $\boldsymbol{\nu}(\bullet,\bullet)\,P$, the two dual endpoints are simultaneously mapped to distinct indices within the continuation of P, namely 0 and 1. For communication prefixes like $x(\bullet).P$, the payload is implicitly bound as index 0 within P, while the explicit free channel x is deliberately preserved as the subject. By maintaining explicit free names strictly for these outward-facing communications, the transition labels and operational semantics of the calculus remain completely unaffected by the architectural shift.

While the locally nameless architecture elegantly resolves α-equivalence, moving between abstracted and instantiated forms requires specialised operations. To execute a substitution or evaluate a process across a binder, we must formally define how to open and close these terms.

3.3 Opening and Closing Binders

Because the locally nameless representation separates free and bound variables, standard substitution cannot be applied uniformly under binders. Instead, moving between bound and free representations is handled by two dual operations, namely *opening* and *closing* [Charguéraud 2012].

Opening (Instantiation). Opening is the operation of traversing into a binding construct and replacing the exposed bound index with a concrete free name. Formally, opening a process P at depth k with a free name x is denoted $\{k \mapsto x\}P$.

The operation proceeds via structural recursion. A depth counter, initially set to 0, tracks the number of binders traversed. As the recursion descends through the syntax tree, the counter increments when passing a new binder. Consider being at depth k, then traversing a single-name binder like $x(\bullet).P$ increments the depth to $k+1$, while traversing a double-name binder like $\nu(\bullet, \bullet)\, P$ would increment it to $k+2$.

When the recursion reaches a bound variable $\mathrm{bound}(n)$, the index n is compared against the current depth k. If $n = k$, the variable refers to the binder currently being opened and is replaced by the free name $\mathrm{free}(x)$. Otherwise, the index refers to a different binder and is left unchanged.

Crucially, when the opening operation encounters an existing free variable $\mathrm{free}(y)$, it simply ignores it. This guarantees that opening is entirely capture-avoiding by construction. In the operational semantics, when a single binder like an input prefix $z(\bullet).P$ receives a channel x, the continuation P is opened as $\{0 \mapsto x\}P$. For a restriction $\nu(\bullet, \bullet)\, P$, the dual endpoints x and y are opened sequentially as $\{0 \mapsto x\}(\{1 \mapsto y\}P)$.

Closing (Abstraction). Closing is the exact inverse of opening. It is the operation of taking an open term and abstracting a specific free name x into a bound index at depth k, denoted $\{k \mapsfrom x\}P$. While recursing, bound indices are ignored as they cannot contain free names. When the recursion encounters a free term like $\mathrm{free}(y)$, it checked if $y = x$. If they match, the free name is replaced by $\mathrm{bound}(k)$.

This operation is primarily used when constructing new binders, such as moving a free channel into a restricted scope. To close multiple free names simultaneously, such as abstracting x and y into a restriction, the operations are applied sequentially, by first closing y at depth $k + 1$, followed by closing x at depth k.

Locally Closed Terms. The separation of names and indices introduces a new syntactic anomaly, namely that a term can contain a *dangling* index that does not point to any enclosing binder. For example, the process term $\mathrm{bound}(0)[\,].\mathbf{0}$ is syntactically valid, matching the syntax for an empty send, but mathematically it is meaningless because the index 0 has no corresponding binder.

To filter out these meaningless terms, the locally nameless architecture introduces the predicate of being *locally closed*, denoted $lc(P)$. A term is locally closed if and only if all of its De Bruijn indices are properly bound by an enclosing constructor.

The locally closed predicate excludes terms containing dangling indices and therefore characterises the well-formed syntax of the calculus. Establishing this predicate is not only a static validation step, but rather it forms a key invariant throughout the mechanisation, which guarantees the soundness of the calculus. Generally, any subsequent definition, such as variable substitution, structural congruence, or operational semantics, must be mathematically proven to *preserve* the locally closed property. In practice, this is achieved through the *regularity of typing*, which requires proving that all valid typing derivations inherently produce locally closed terms. Consequently, operations proven to preserve typing will naturally carry this invariant as a structural guarantee.

3.4 Co-finite Quantification

With the syntax partitioned into names and indices, and the mechanics of opening and closing established, the final theoretical hurdle is defining how inductive relations, such as typing judgements and labelled transitions, operate across binders.

In paper-based proofs, when a rule requires opening a binder, Barendregt's variable convention allows the author to simply state: *pick a fresh name x*. In a proof assistant, this informal generation of a fresh name must be encoded into the constructors of the inductive relations. Two naive approaches to bind this fresh name, both of which introduce severe mechanical flaws [Aydemir et al. 2008; Charguéraud 2012]:

(1) **Existential Quantification ($\exists x \notin \mathrm{f}(P)$):** The rule requires the property to hold for at least one fresh name. While this makes proof construction straightforward, it yields weak induction hypotheses, since induction only provides the property for one particular fresh name chosen during the original derivation.
(2) **Universal Quantification ($\forall x$):** The rule requires the property to hold for every name. This provides a strong induction hypothesis, but is too restrictive for proof construction, since the property cannot generally be shown for names that already occur free in the surrounding context.

To solve this dilemma, the locally nameless architecture employs **co-finite quantification** [Aydemir et al. 2008; Charguéraud 2012]. Rather than quantifying over one name or all names, the inductive rules quantifies universally over all names *except* those not in an arbitrary, finite set L. Formally, a rule requiring a fresh name uses the premise: $\forall x \notin L$.

By deferring the exact choice of the forbidden set L to the user at the time of proof, co-finite quantification achieves the best of both worlds. When constructing a proof tree, the user can instantiate L as the set of all free variables currently in the environment, process, and transition labels ($\mathrm{f}(P) \cup \mathrm{f}(\Gamma) \cup \ldots$). This ensures the premise is trivially satisfiable. Conversely, when performing structural induction, the co-finite universal quantifier guarantees that the induction hypothesis applies to *any* fresh name the user could possibly need, provided they pick one outside the finite set L.

Application to πLL Types. Although type variables are represented using the locally nameless separation of free and bound constructors, their metatheory is handled using depth-indexed reasoning rather than cofinite quantification. Because type variables are static and do not dynamically communicate across parallel scopes like channels do, managing freshness sets L for them introduces unnecessary proof overhead.

Instead, the system evaluates polymorphic typing rules using a purely depth-indexed approach, using a De Bruijn shifting operator. We define the shifting of a type A by a distance d at a cutoff depth c, denoted mathematically as $\uparrow^c_d A$. Using this infrastructure, consider the typing rule for the universal type input ($\forall.B$):

$$\frac{n+1 \vdash P :: \uparrow^1_0 \Gamma, x : B}{n \vdash x(\diamond).P :: \Gamma, x : \forall.B}\ \forall_{\mathrm{DB}}$$

Rather than opening the binder with a fresh explicit name, the typing judgment simply increments a depth counter $n+1$, legally bringing index 0 into scope. To maintain correct index distances, the typing environment Γ is explicitly shifted by a distance of 1 at cutoff 0 ($\uparrow^1_0 \Gamma$), incrementing all free type indices within the context to prevent accidental capture.

Application to πLL Channels. This co-finite quantification strategy fundamentally reshapes the inductive definitions for process channels. For example, consider the typing rule that introduces the prefix $x(\bullet).P$. Under a paper-based presentation, the premise implicitly requires a fresh session name y. Under our mechanised architecture, the rule uses co-finite quantification and the opening operation:

$$\frac{\forall y \notin L, \quad n \vdash \{0 \mapsto y\}P :: \Gamma, x : B, y : A}{n \vdash x(\bullet).P :: \Gamma, x : A \parr B}\ \parr_{\mathrm{LN}}$$

Here, the rule states that the process is well-typed if, for any fresh channel variable y outside some finite set of forbidden variables L, the opened process P is well-typed under the environment extended with the opened payload. By utilizing this mixed evaluation strategy, co-finite quantification for dynamic channels, and depth-indexed evaluation for static types, the architecture minimizes bureaucratic renaming overhead while maintaining a unified underlying syntax.

With this mathematical foundation established, the conceptual theory of πLL is fully prepared for implementation. The following chapter details how this locally nameless architecture is concretely mechanised in the Lean 4 proof assistant.

The corresponding Lean definitions for channels, opening and closing, and selected local-closure predicates are included in ??; the full development is available in the accompanying codebase.

4 Formalisation

This section describes the mechanisation of πLL in Lean 4, bridging the theoretical foundations developed in Sections 2 and 3 and their concrete implementation. The formalisation is organised into three top-level namespaces: `PiLL.Model`, which defines the static structure of the calculus, `PiLL.Semantics`, which defines its operational behaviour, and `PiLL.SessionFidelity`, which establishes the metatheoretic results connecting the two. Each component is presented in dependency order, highlighting design decisions and divergences from Montesi and Peressotti [2021].

4.1 Model

The `Model` namespace formalizes the static structure of the full πLL calculus and is organized around the layered architecture introduced in Section 2. The mechanisation proceeds bottom-up: session types, defined in `STypes/`, provide the binding structure used by processes in `Processes/`, processes are then typed relative to environments and hyperenvironments from `Environments/` and `HyperEnvironment/`, finally, typing derivations and their metatheoretic infrastructure are collected in `Judgement/`. This includes inversion lemmas, linearity properties, substitution principles, and the co-finite freshness machinery described in Section 3.4. Shared syntactic infrastructure used across these layers is provided by `Base.lean`.

4.1.1 Session Types. Session types are defined in `STypes/Basic.lean` as the inductive datatype `Types`. Following the locally nameless syntax established in Section 3.2, type variables are isolated into a separate inductive type, `TVar`, which explicitly distinguishes between free and bound occurrences:

```
inductive TVar : Type
  | free  (x : FTVar)
  | bound (x : BTVar)
```

Here, `FTVar` represents a free type variable identified by a natural number, while `BTVar` represents a bound type variable encoded as a De Bruijn index.

The remaining constructors directly mirror the theoretical grammar of the calculus. In addition, the formalisation introduces uninterpreted atomic types (`atom` and `atomDual`), representing protocol leaves not further decomposed by the session type grammar and providing the base cases for the inductive structure of the syntax. Table 3 illustrates a representative correspondence between the paper syntax, the locally nameless representation, and the Lean implementation.

Notably, the polymorphic binders $\forall.A$ and $\exists.A$ reflect the depth-indexed approach detailed in Section 3.4. The constructors keep the bound De Bruijn index implicit, taking only the continuation body `A` as an argument (an underscore "_" has been appended to the constructor names to prevent namespace collisions with Lean's built-in `forall` and `exists` keywords).

Table 3. Comparison of standard, locally nameless, and Lean representations for πLL types.

Standard	Locally Nameless	Lean Constructor	Binds
$A \otimes B$	$A \otimes B$	`| tensor (A B : Types)`	None
$\mathbf{1}$	$\mathbf{1}$	`| one`	None
$\forall X.A$	$\forall.A$	`| forall_ (A : Types)`	1 type
$\exists X.A$	$\exists.A$	`| exists_ (A : Types)`	1 type
X	X	`| var (v : TVar)`	None

Duality. `Types.dual` implements duality as a structurally recursive function, mapping each constructor to its dual. For example:

```
def Types.dual : Types -> Types
  | .atom n      => .atomDual n
  | .atomDual n => .atom n
  | .tensor A B  => .parr    (dual A) (dual B)
  | .forall_ A   => .exists_ (dual A)
  | .one         => .bot
  | ...
```

The fundamental property that duality is an involution, $(A^{\perp})^{\perp} = A$, is proved in `STypes/Lemmas.lean` by structural induction.

The formalisation additionally establishes that duality preserves local closure and commutes with both shifting and substitution:

$$A.\mathtt{lc} \iff A^{\perp}.\mathtt{lc} \qquad \uparrow_d^c (A^{\perp}) = (\uparrow_d^c A)^{\perp} \qquad B^{\perp}\{A/k\} = (B\{A/k\})^{\perp}$$

Local closure. `STypes/Properties.lean` defines local closure for types as a depth-indexed predicate `Types.lc (k : Nat) : Types` $\rightarrow$ `Prop`, where the depth parameter k tracks how many binders the recursion has traversed.

At the leaves of the syntax tree, local closure is evaluated by the helper predicate `TVar.lc`. Because free variables are explicit they do not contain any indices, as such they are inherently locally closed at any depth. By contrast, a bound variable is only locally closed if its De Bruijn index i is strictly less than the current depth k, ensuring it points to an enclosing scope:

```
def TVar.lc : Nat -> TVar -> Prop
  | _, .free _   => True
  | k, .bound i  => i < k
```

The primary `Types.lc` predicate is then defined by structural recursion. For non-binding constructors, the depth parameter distributes across the sub-terms. When a variable is encountered, the predicate delegates the check to `TVar.lc`. When descending into the body of a polymorphic binder, the depth counter increments by one, bringing the newly introduced index safely into scope:

```
def Types.lc : Nat -> Types -> Prop
  | _, .one         => True
  | k, .var v       => TVar.lc k v
  | k, .tensor A B  => A.lc k ∧ B.lc k
  | k, .forall_ A   => A.lc (k + 1)
```

```
  | k, .exists_ A   => A.lc (k + 1)
  | ...
```

Finally, a type is defined as *closed* if it contains no dangling indices at the top level, meaning it is locally closed at depth 0. This is registered as the abbreviation `Types.lc_0`.

Shifting. `STypes/Shift.lean` defines shifting following the standard De Bruijn shifting operation introduced in Section 3.4. The operation traverses the syntax tree structurally using a cutoff depth d and a shift amount c.

For standard connectives, shifting distributes recursively across subterms, while base constructors such as `one` and `bot` remain unchanged. When traversing a polymorphic binder like `forall_`, the cutoff depth increments by one to account for the newly entered scope:

```
def Types.shift (d c : Nat) : Types -> Types
  | .var v        => .var (v.shift d c)
  | .tensor A B  => .tensor (A.shift d c) (B.shift d c)
  | .forall_ A   => .forall_ (A.shift (d + 1) c)
  | t            => t
```

At the variable level, free variables remain unchanged. For bound indices, the function compares the De Bruijn index i against the cutoff depth d. Indices satisfying $i < d$ are locally bound within the traversed scope and therefore remain unchanged. Otherwise, the index is incremented by c to preserve its reference to the surrounding environment:

```
def TVar.shift (d c : Nat) : TVar -> TVar
  | .bound i => if i < d then .bound i else .bound (i + c)
  | .free n  => .free n
```

Substitution. `STypes/Subst.lean` defines substitution as the replacement of a target De Bruijn index k in a host type B by a payload type A, denoted $B\{A//k\}$.At the variable level, the function compares a bound De Bruijn index i with the target depth k. If $i = k$, the target index has been reached and the variable is replaced by A. If $i < k$, the variable is bound by an inner scope and is left unchanged. Finally, if $i > k$, the index refers to an outer binder beyond the substituted variable. Since substitution removes the binder corresponding to k, such indices are decremented by one to preserve their relative distance to the surrounding environment.

```
def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.free n)  => .var (.free n)
  | .var (.bound i) =>
      if i == k then A
      else if i > k then .var (.bound (i - 1))
      else .var (.bound i)
  | .tensor A B => .tensor (A.subst A k) (B.subst A k)
```

The operation is lifted to the full `Types` syntax by structural recursion. For ordinary connectives, substitution distributes across subterms. When descending into a polymorphic binder such as `forall_`, the target depth is incremented to $k + 1$, and the payload type A is shifted by one. This prevents the free indices in A from being captured by the newly entered binder:

```
  | .forall_ B => .forall_ (B.subst (shift 0 1 A) (k + 1))
```

The interaction between substitution and local closure is captured by `Types.lc_subst_inv`, which states that substituting a type A locally closed at depth $n+k$ for index k in B preserves local closure: $(B\{A/k\}).\texttt{lc}(n+k) \iff B.\texttt{lc}(n+k+1)$. Intuitively, substituting for index k removes one binder from B, so local closure at $n+k$ in the substituted term corresponds to local closure at $n+k+1$ in the original. The specialisation `Types.lc_subst_inv_0` fixes $k=0$, which is the form later used in the proof of `Typing_preserves_lc`.

Notation. `STypes/Notation.lean` introduces notation mirroring the mathematical presentation from Section 2.3. The only divergence is that exponentials are written $!!A$ and $??A$ to avoid clashes with Lean syntax.

4.1.2 Processes. Processes are defined in `Processes/Basic.lean` as the inductive type `Proc`. Following the established locally nameless architecture, channel references are partitioned into a dedicated `Channel` inductive. Explicit free channels are instantiated as concrete natural numbers (`FPName`). While the raw process syntax permits arbitrary reuse of these numbers, using `Nat` allows the formalisation to systematically compute strictly fresh names by incrementing the supremum of any finite set of existing names (mechanised in `Processes/Fresh.lean`). Conversely, bound channels are represented using De Bruijn indices, encoding their relative distance to the corresponding binder.

```
inductive Channel : Type where
  | free  (x : FPName)
  | bound (x : BPName)
```

As discussed in Section 3.2, binding constructs in the process syntax no longer carry explicit payload variables. Instead, bound occurrences are captured purely by their relative position. Comparing the standard paper grammar with the intermediate locally nameless notation and the final constructors of the `inductive Proc` type illustrates this abstraction:

Standard	**Locally Nameless**	**Lean Constructor**	**Binds**
$x[y].P$	$x[\bullet].P$	`| tensor (x : Channel) (P : Proc)`	1 channel
$x(y).P$	$x(\bullet).P$	`| parr (x : Channel) (P : Proc)`	1 channel
$\nu xy\,P$	$\nu(\bullet,\bullet)\,P$	`| cut (P : Proc)`	2 channels
$x(X).P$	$x(\diamond).P$	`| input (x : Channel) (P : Proc)`	1 type
$x[\textsc{dup}](x').P$	$x[\textsc{dup}](\bullet).P$	`| duplicate (x : Channel) (P : Proc)`	1 channel

Notice that `cut` takes only a continuation P with no explicit channel arguments, as both dual endpoints are simultaneously bound as indices 0 and 1 within the scope of P. Similarly, for constructs like `tensor`, `parr`, and `input`, the subject channel x over which the interaction occurs is kept explicit, while the transmitted payload is anonymized into the continuation. The full inductive definition is listed in Section D.2.

Opening and closing. `Processes/OpenClose.lean` defines opening and closing operations following the locally nameless principles established in Section 3.3. In the theoretical presentation, opening a process at depth k with x is denoted as $\{k \mapsto x\}P$, and conversely, abstracting a specific name back into a bound index is denoted as $\{k \mapsfrom x\}P$.

To mechanise this in Lean, we denote explicit free channels as $\#x$ (mapping to `Channel.free x`) and introduce custom notation for these substitution operations. Expressions such as $P(\!(k|\#x)\!)$ and $P(\!(k|\#x,\#y)\!)$ denote the opening of a process by instantiating the k-th (and $k+1$-th) bound indices with free channels. Conversely, $P\langle\!\langle k|\#x\rangle\!\rangle$ and $P\langle\!\langle k|\#x, \#y\rangle\!\rangle$ denote the closing of a process by abstracting x and y back into De Bruijn indices. For convenience, the depth parameter can be omitted when operating at the innermost binder ($k = 0$), simplifying the notation to $P(\!(\#x, \#y)\!)$ and $P\langle\!\langle \#x, \#y\rangle\!\rangle$.

Because binders are represented positionally, the depth parameter k increments according to the number of channels bound by each constructor. For standard communication prefixes like `tensor`, `parr`, and the server `duplicate` operation, exactly one channel is bound, incrementing k by one. The restriction constructor (`cut`) simultaneously binds two dual endpoints, requiring k to increment by two. The `input` constructor binds a type variable rather than a channel and therefore leaves k unchanged. The `server` constructor additionally carries an explicit set of dependency channels `zs : Finset Channel`, representing the replicable processes the server depends on, which must be acted on simutaneously with x and P.

```
def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
  match P with
  | .tensor x P     => .tensor (x.open u k) (P.open u (k + 1))
  | .parr   x P     => .parr   (x.open u k) (P.open u (k + 1))
  | .cut      P     => .cut                 (P.open u (k + 2))
  | .duplicate x P  => .duplicate (x.open u k) (P.open u (k + 1))
  | .server x zs P  => .server (x.open u k) (zs.open u k) (P.open u k)
  | .input  x P     => .input  (x.open u k) (P.open u k)
  | ...
```

At the leaves of the syntax tree, this recursive traversal delegates to the base case defined by `Channel.open` and `Finset.open`. For bound channels, the De Bruijn index i is compared against the target depth k and replaced with v upon an exact match. For dependency sets, `Channel.open` is mapped pointwise. In both cases free channels remain entirely unaffected:

```
def Channel.open (u v : Channel) (k : Nat) : Channel :=
  match u with
  | bound i => if i == k then v else bound i

def Finset.open (zs : Finset Channel) (v : Channel) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.open v k)
```

For restriction the `HasOpenTwo` typeclass from `Base.lean` is instantiated by opening sequentially: first replacing index $k + 1$ with the second endpoint v, then replacing index k with the first endpoint u:

```
instance : HasOpenTwo Proc Channel Channel Nat where
  open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)
```

To complete the bidirectional infrastructure, `Proc.close`, `Channel.close`, and `Finset.close` define the corresponding closing operation. `Proc.close` traverses the process tree using a structural recursion identical to `Proc.open`, applying the same depth-incrementing rules. At the leaves, `Channel.close` compares each free channel against the target name x and abstracts it into a bound De Bruijn index at depth k upon a match, while bound indices are left unchanged. Dependency sets are handled pointwise by `Finset.close`:

```
def Channel.close (u : Channel) (x : FPName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i
```

```
def Finset.close (zs : Finset Channel) (x : FPName) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.close x k)
```

```
def Channel.close (u : Channel) (x : FPName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i

def Finset.close (zs : Finset Channel) (x : FPName) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.close x k)
```

Local closure. `Processes/LocalClosure.lean` defines local closure for processes as a two-parameter predicate `Proc.lc (k n : Nat) : Proc → Prop`. Because the calculus features a two-sorted syntax, the predicate must maintain independent depth counters: k tracks the depth of channel binders, while n tracks the depth of type binders. The type depth is needed when a process embeds a type, for instance as the payload of an output prefix, since the embedded type must be checked relative to the current type-binder depth.

The structural recursion evaluates each constructor by incrementing the appropriate depth counter according to the number and sort of binders introduced. Channel binders increase the channel depth k, while type binders increase the type depth n. For example, `tensor` binds a single continuation channel, incrementing the depth to $k + 1$, whereas `cut` binds two dual endpoints simultaneously, incrementing the depth to $k + 2$. Conversely, `input` binds a type variable, leaving k unchanged while incrementing n to $n + 1$. The `server` constructor therefore requires local closure of `x`, `zs`, and `P`. Constructors introducing no binders, leave both counters unchanged

```
def Proc.lc : Nat -> Nat -> Proc -> Prop
  | _, _, .nil             => True
  | k, n, .one x P         => x.lc k ∧ P.lc k n
  | k, n, .tensor x P      => x.lc k ∧ P.lc (k + 1) n
  | k, n, .cut P           => P.lc (k + 2) n
  | k, n, .input x P       => x.lc k ∧ P.lc k (n + 1)
  | k, n, .duplicate x P   => x.lc k ∧ P.lc (k + 1) n
  | ...
```

At the leaves, `Channel.lc` checks that any bound index is strictly within the current depth, while free channels are trivially locally closed. For dependency sets, `Finset.lc` requires every channel in the set to satisfy `Channel.lc k`:

```
def Channel.lc : Nat -> Channel -> Prop
  | _, .free _    => True
  | k, .bound i   => i < k

def Finset.lc (zs : Finset Channel) (k : Nat) : Prop :=
  ∀ z ∈ zs, z.lc k
```

A process is considered globally *closed* when it contains no dangling indices of either sort, meaning `Proc.lc 0 0` holds. This is registered as the abbreviation `Proc.lc_0`.

Free names and freshness. `Processes/Names.lean` defines `Proc.f : Proc → Finset FPName` to compute the free channel names of a process. The function traverses the syntax tree structurally, collecting all explicit free channel identifiers into a finite set. Since locally nameless binders are represented using anonymous indices, bound channels contribute nothing to this set.

For constructors carrying explicit subject channels, such as `tensor` and `link`, the free names of those channels are combined with the recursively computed free names of their continuations. Constructors without explicit channel identifiers of their own, such as `cut`, contribute only the free names recursively obtained from their continuations. The `server` constructor contributes the free names of its subject channel, its dependency set and continuation:

```
def Proc.f : Proc -> Finset FPName
  | .cut P          => P.f
  | .par P Q        => P.f ∪ Q.f
  | .link x y       => x.f ∪ y.f
  | .tensor x P     => x.f ∪ P.f
  | .one x P        => x.f ∪ P.f
  | .server x zs P  => x.f ∪ zs.f ∪ P.f
  | .nil        => {}
  | ...
```

At the leaves, `Channel.f` returns the singleton set for free channels and the empty set for bound indices. For dependency sets, `Finset.f` collects free names across all channels in the set via `biUnion`:

```
def Channel.f : Channel -> Finset FPName
  | .free x   => {x}
  | .bound _ => {}

def Finset.f (zs : Finset Channel) : Finset FPName :=
  zs.biUnion Channel.f
```

The resulting finite set is used by `Processes/Fresh.lean` to generate names outside a forbidden context. Since free process names are natural numbers, a fresh name for a finite set L is computed as `freshName L` $= \sup(L) + 1$, with `fresh_is_fresh` confirming that every element of L is strictly smaller than this value. The lemmas `exists_one_fresh` and `exists_two_fresh` then guarantee one or two distinct names outside L, providing witnesses for the co-finite quantification premises used throughout the typing and transition rules, in particular for restriction, where two fresh and distinct endpoints must be introduced simultaneously.

Substitution. The process syntax supports two substitution operations, reflecting its two-sorted binding structure. Name substitution is defined in `Processes/SubstNames.lean` as the function `Proc.substNames (R T : FPName) : Proc` $\longrightarrow$ `Proc`, replacing all explicit free occurrences of the target name T with the replacement name R. Since bound channels are represented by De Bruijn indices, name substitution only acts on explicit free channel leaves and leaves binders untouched. Consequently, the operation is capture-avoiding by construction and does not require α-renaming or additional freshness side conditions.

Name substitution is defined in `Processes/SubstNames.lean` as `Proc.substNames (R T : FPName) : Proc` $\longrightarrow$ `Proc`, replacing all explicit free occurrences of T with R. At the leaves, `Channel.subst` replaces a matching free channel and leaves bound indices unchanged; dependency sets are handled pointwise by `Finset.subst`:

```
def Channel.subst (R T : FPName) : Channel -> Channel
  | .free x   => if x == T then .free R else .free x
  | .bound i => .bound i
```

```
def Finset.subst (zs : Finset Channel) (R T : FPName) : Finset Channel :=
  zs.image (fun u => u.subst R T)
```

Type substitution is defined in `Processes/SubstTypes.lean` as `Proc.substTypes (A : Types) (k : Nat) : Proc` $\longrightarrow$ `Proc`, substituting A for the De Bruijn type index k throughout a process. Whenever the traversal encounters an embedded type, such as the payload of an output prefix, it delegates to the type-level substitution operation `Types.subst`. When crossing an `input` prefix, which binds a type variable, the target depth is incremented and the substituted type is shifted, mirroring the binder case of `Types.subst` from Section 4.1.1:

```
def Proc.substTypes (A : Types) (k : Nat) : Proc -> Proc
  | .input x P => .input x (P.substTypes (A.shift 0 1) (k + 1))
```

A consequence of this representation is that no analogous channel shifting operation is required. In a pure De Bruijn encoding, variables are represented as relative indices and must be shifted when crossing binders. Free channels, however, are represented as global identifiers rather than relative positions, so their representation is unaffected by binder depth. This avoids structural adjustment of channel names during substitution and simplifies the process-level infrastructure.

Type shifting. `Processes/SubstTypes.lean` also defines `Proc.shiftTypes (d c : Nat) : Proc` $\longrightarrow$ `Proc`, which lifts the type-level shifting operation to processes. The traversal is uniform across most constructors, delegating to `Types.shift` whenever an embedded type is encountered, such as the payload of an `output` prefix. The only non-trivial case is `input`, which binds a type variable and therefore increments the cutoff depth d before traversing its continuation, mirroring the binder case of `Types.shift`:

```
def Proc.shiftTypes (d c : Nat) : Proc -> Proc
  | .output x P A  => .output x (P.shiftTypes d c) (A.shift d c)
  | .input x P     => .input x (P.shiftTypes (d + 1) c)
  | ...
```

Structural Properties. `Processes/Lemmas.lean` establishes the fundamental structural properties required by the later typing and operational metatheory. The lemmas fall into three groups.

The first group supports the typing invariants. `Proc.f_open_erase` and `Proc.f_open_two_erase` establish that opening a process with a fresh name and then erasing that name from the free name set recovers the original free names. These are used in the proof of `Typing_f_eq_names`. `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` show that opening a locally closed abstraction with a locally closed name yields a locally closed process, supporting the proof of `Typing_preserves_lc` showing that well-typed processes are locally closed.

The second group supports the substitution lemmas. `Proc.open_substNames_comm` and `Proc.openCut_subs` establish that opening and name substitution commute, while the corresponding lemmas for type substitution (`Proc.open_substTypes_comm` and `Proc.openCut_substTypes_comm`) ensure that substituting under a binder produces the same result regardless of order. These are essential in the typability proof, where co-finite witnesses from the typing rules must be reconciled with the concrete names introduced by process transitions.

The third group supports freshness reasoning. `Proc.not_mem_f_open` states that a name fresh for both a process and a channel does not appear in the opened process, providing the freshness conditions needed in the typability cases. These results provide the core infrastructure required for the transition semantics developed in subsequent chapters.

4.1.3 *Environments.*

Environments are defined in `Environment/Basic.lean` as `Env`, an abbreviation for a list of channel-type pairs:

```
abbrev Elem := (FPName × Types)
abbrev Env  := List Elem
```

The choice of `List` as the underlying representation deserves explanation. Theoretical environments are unordered and duplicate-free, suggesting `Finset` as a natural implementation. However, since `Finset` is implemented as a quotient over lists, it complicates structural induction, pattern matching, and inversion over environments. This proved inconvenient in a development where typing derivations are frequently inverted and typing environments must be rearranged explicitly. A second attempt using association lists (`AList`) encountered similar proof-engineering difficulties. The final design therefore uses plain lists and recovers the unordered structure through an explicit permutation relation, trading quotient reasoning for structural induction and simpler interaction with Lean's tactic machinery.

Recovering the PCM structure. Merging two environments is implemented as list concatenation, making it a simple structural, but total, function:

```
abbrev Env.merge (Γ Δ : Env) : Env := Γ ++ Δ
infixl:69 "," => Env.merge  -- uses \glq to avoid clashing with the actual comma
```

Because lists do not naturally form a Partial Commutative Monoid (PCM) but are associative by definition, we only need to formally recover commutativity.

Since list concatenation is associative definitionally but not commutative, the commutative structure of environments is recovered by reasoning up to permutation. This is achieved using Lean's built-in `List.Perm` relation, instantiated through a custom `HasPerm` typeclass and denoted by the infix relation $\Gamma \sim \Delta$.

The essential monoidal laws, including commutativity, associativity, and left and right identity for $\varnothing$,[1] are established as:

```
lemma Env.merge_comm (Γ Δ : Env) : Γ,Δ ∼ Δ,Γ
lemma Env.merge_assoc (Γ Δ Ξ : Env) : Γ,Δ,Ξ = Γ,(Δ,Ξ)
```

The partiality of environment merging is enforced by explicit predicates rather than by making `Env.merge` a partial function. `Env.names` extracts the domain as a `Finset`, enabling Lean's built-in `Disjoint` typeclass. Internal linearity (no channel typed twice) is enforced by `Env.Nodup`:

```
def Env.names (Γ : Env) : Finset FPName := (Γ.map Prod.fst).toFinset
def Env.disjoint (Γ Δ : Env) : Prop := Disjoint Γ.names Δ.names
def Env.Nodup (Γ : Env) : Prop := (Γ.map Prod.fst).Nodup
```

The relationship between linearity, disjointness, and merging of typing environments is captured by `Env.Nodup_merge_iff`, which states that a merged environment is linear if and only if both component environments are individually linear and mutually disjoint.

Server usability. The typing rule for server replication requires all dependency channels to carry exponential request types. This side condition is captured by `Env.serverUsable`, which asserts that every type in an environment satisfies `Types.isServerUsable`, i.e. has the form $?B$.

[1]Using $\varnothing$ rather than the literal `[]` requires explicit rewrite lemmas for terms such as $\varnothing, \Gamma$. If `[]` were used directly, $[\,], \Gamma$ would reduce by definitional equality, since `Env.merge` abbreviates `List.append`.

```
def Env.serverUsable (Γ : Env) : Prop :=
  ∀ p, p ∈ Γ -> p.snd.isServerUsable
prefix:max "?ₑ" => Env.serverUsable
```

Lifted operations. Since environments contain types, the locally nameless operations from Section 4.1.1 are lifted pointwise over the type component of each `Elem`. Local closure, type shifting, name substitution, and type substitution are all defined by mapping over the list. The file `Environment/Lemmas.lean` proves that these operations preserve the names, disjointness, nodup, and permutation properties of environments.

The interaction between shifting and local closure is captured by `Env.lc_shift_inv`, which states that shifting an environment by one preserves local closure: $(\Gamma^{+}).\mathtt{lc}(n{+}k{+}1) \iff \Gamma.\mathtt{lc}(n{+}k)$. Shifting increments all free type indices, so an environment locally closed at depth $n+k$ becomes locally closed at depth $n{+}k{+}1$ after shifting, and vice versa. The specialisation `Env.lc_shift_inv_0` is used in the `forall_` case of `Typing_preserves_lc` to recover local closure of the unshifted environment from that of the shifted one.

4.1.4 Hyperenvironments. Hyperenvironments are defined in `HyperEnvironment/Basic.lean` as `HyperEnv`, an abbreviation for a list of environments:

```
abbrev HyperEnv := List Env
abbrev HyperEnv.merge (𝒢 ℋ : HyperEnv) : HyperEnv := 𝒢 ++ ℋ
infixl:55 " |ₕ " => HyperEnv.merge
```

As for environments, merging is implemented as list concatenation, using $\varnothing$ as the unit and $\mathcal{G}\,|_h\,\mathcal{H}$ as notation for composition. Associativity and unit laws therefore follow from the underlying list structure. However, recovering the commutative structure requires more than the ordinary list permutation relation, since hyperenvironments are lists whose elements are themselves environments considered modulo permutation.

Deep permutation. For environments, Lean's `List.Perm` relation is sufficient, since it allows arbitrary reordering of channel-type pairs. For hyperenvironments, however, permutation must operate at two levels: it must allow reordering of component environments, and it must also respect permutation equivalence inside each component environment.

For example, consider the hyperenvironment containing a single component environment $[x : A, y : B]$. Ordinary `List.Perm` treats this component environment as an opaque element, so it does not identify it with $[y : B, x : A]$. The custom relation `HyperEnv.Perm` instead permits component environments to be replaced by permutation-equivalent environments.

```
inductive HyperEnv.Perm : HyperEnv -> HyperEnv -> Prop where
  | nil  : Perm [] []
  | cons : {Γ Δ : Env} {𝒢 ℋ : HyperEnv} : Γ ~ Δ -> Perm 𝒢 ℋ -> Perm (Γ :: 𝒢) (Δ :: ℋ)
  | swap : {Γ Δ : Env} {𝒢 : HyperEnv} : Perm (Γ :: Δ :: 𝒢) (Δ :: Γ :: 𝒢)
  | trans {𝒢 ℋ 𝒥 : HyperEnv} : Perm 𝒢 ℋ -> Perm ℋ 𝒥 -> Perm 𝒢 𝒥
```

The key constructor is `cons`. Unlike ordinary list permutation, it does not require the head environments to be syntactically identical, it only requires them to be permutation-equivalent. Thus `HyperEnv.Perm` is a deep permutation relation, propagating equivalence both across the ordering of component environments and within the environments themselves.

Well-formedness. Hyperenvironment well-formedness is decomposed into two predicates. The predicate `HyperEnv.Nodup` requires each component environment to be internally linear, while `HyperEnv.PairwiseDisjoint` requires distinct component environments to have disjoint domains:

```
def HyperEnv.Nodup (𝒢 : HyperEnv) : Prop :=
  ∀ Γ, Γ ∈ 𝒢 -> Env.Nodup Γ

def HyperEnv.PairwiseDisjoint (𝒢 : HyperEnv) : Prop :=
  List.Pairwise Env.disjoint 𝒢
```

The distinction is important because the two predicates rule out different violations of linearity. For example, assuming x, y, and z are distinct names:

- $[x : A] \parallel_h [y : B, z : C]$ satisfies both predicates.
- $[x : A, x : B] \parallel_h [y : C]$ violates `Nodup`, since x appears twice within one component environment.
- $[x : A] \parallel_h [x : B, y : C]$ violates `PairwiseDisjoint`, since x appears in two distinct components.

Together, these predicates enforce global linearity: every channel appears at most once across the whole hyperenvironment.

Lifted operations. The operations on environments are lifted once more to hyperenvironments by applying them pointwise to each component environment. Thus, type shifting, name substitution, and type substitution on hyperenvironments follow the same pattern as the corresponding operations on environments. The file `HyperEnvironment/Lemmas/Basic.lean` proves that these lifted operations preserve the structural invariants used later in the development, including names, disjointness, pairwise disjointness, and deep permutation. For example, preservation of deep permutation under shifting is proved by induction on the proof of the hyperenvironment permutation relation, using the corresponding environment-level permutation lemmas. Since these definitions are obtained uniformly from the environment-level construction, they are not reproduced in full here. The complete definitions and proofs are included in the accompanying Lean archive.

4.1.5 Typing Judgements. The typing relation is defined in `Judgement/Basic.lean` as the inductive predicate `Typing`:

```
inductive Typing : Nat -> Proc -> HyperEnv -> Prop
```

We write $n \vdash P :: \mathcal{G}$ for a derivation of this predicate. The index n tracks the current type-binder depth, while the remaining indices record the process being typed and its associated typing environment. This intrinsic indexing is later used by the erasure maps in the semantics and metatheory.

Type-depth indexing. The type-depth index is relevant whenever the derivation crosses a polymorphic binder. For example, the `forall_` constructor increments the depth from n to $n + 1$ and shifts the surrounding environment to keep embedded type indices in scope:

```
| forall_ :
    Typing (n + 1) P [x:B :: Γ⁺] ->
    Typing n (#x⦅◇⦆.P) [x:∀.B :: Γ]
```

The shift Γ^+ abbreviates shifting the type indices in Γ by one starting at depth 0, mirroring the binder-sensitive shifting used by type substitution in Section 4.1.1.

The depth index is also relevant for the MIX rule, which requires both sub-derivations to share the same depth n. When derivations are available at different depths, they can be brought into alignment using `Typing_weakening`. This lemma states that a derivation at depth n can be lifted to depth $n + c$ by shifting the embedded type indices in both the process and the hyperenvironment:

```
lemma Typing_weakening {n : Nat} {P : Proc} {𝒢 : HyperEnv} :
  Typing n P 𝒢 -> ∀ d c, Typing (n + c) (P ↑ d, c) (𝒢 ↑ d, c) := by ...
```

This provides the weakening principle needed to compose derivations across polymorphic binders while keeping embedded type indices in scope.

Exchange and explicit side conditions. Because environments and hyperenvironments are represented as lists, the unordered and partial structure of the theoretical hyperenvironment is recovered explicitly. The typing relation includes exchange rules for reasoning up to permutation equivalence:

```
| exchange_env   : Typing n P (Γ :: 𝒢) -> Γ ~ Δ -> Typing n P (Δ :: 𝒢)
| exchange_hyper : Typing n P 𝒢 -> 𝒢 ~ ℋ -> Typing n P ℋ
```

The exchange rules recover the unordered character of the paper presentation without quotienting environments or hyperenvironments. The complementary issue is partiality: although merging is implemented as total list concatenation, typing rules carry explicit disjointness and freshness premises to ensure that only valid linear compositions are formed. For example, `mix` requires the two hyperenvironments to be disjoint, while prefix rules such as `tensor` and `parr` require freshness premises ensuring that subject channels do not already appear in the continuation environment.

The same design principle reappears later in the operational semantics, where transitions must also respect permutation equivalence of environments and hyperenvironments.

Co-finite quantification. Binding rules use co-finite quantification as described in Section 3.4. For example, the `parr` constructor types an input prefix by requiring the opened continuation to be well typed for every fresh payload name outside a finite set L:

```
| parr (hF : x ∉ Γ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P⦅#y⦆) [y:A :: x:B :: Γ]) ->
    Typing n (x⦅•⦆.P) [x:A ⅋ B :: Γ]
```

Here, the process carries no explicit payload name; the bound occurrence in the continuation is instantiated by opening P with the fresh name y. The premise `hF` enforces that the subject channel x is not already present in the continuation environment. Rules binding multiple names, such as `cut`, follow the same pattern but quantify over two distinct fresh names.

Structural invariants. `Judgement/Names.lean` establishes the central name invariant for the typing relation, named `Typing_f_eq_names`. It states that, for any well-typed process, the free names of the process coincide exactly with the domain of its hyperenvironment.

$$n \vdash P :: \mathcal{G} \implies P.f = \mathcal{G}.names$$

The proof proceeds by induction on the typing derivation. Most cases follow by unfolding `Proc.f` and `HyperEnv.names` and rewriting with the induction hypothesis. In the co-finite binding cases, the proof instantiates fresh names, applies the induction hypothesis to the opened process, and then uses `Proc.f_open_erase` to cancel the introduced names from the free name set and recover equality for the closed process.

This ensures exact correspondence between process free names and typed environment entries: no free process channel is left untyped, and no environment entry is unused.

Building on this, `Judgement/Properties/Linearity.lean` proves that any well-typed hyperenvironment is pairwise disjoint and all component environments have no duplicate names.

$$n \vdash P :: \mathcal{G} \implies \texttt{Nodup}\ \mathcal{G} \wedge \mathcal{G}.\texttt{PairwiseDisjoint}$$

The proof proceeds by induction on the typing derivation. Most non-binding rules are discharged uniformly by unfolding the relevant definitions and applying the induction hypothesis. The co-finite binding cases (`cut`, `tensor`, `parr`, and contraction `c`) require instantiating fresh names and reconstructing both nodupness and disjointness of the component environments from the opened induction hypothesis. The exchange cases use preservation of these properties under permutation.

Together, `Typing_f_eq_names` and `Typing_preserves_linearity` guarantee that typing derivations can only be constructed for well-formed, linear contexts. In particular, a well-typed process cannot contain duplicate free channel occurrences: by the name correspondence, duplicate free names would require duplicate entries in the hyperenvironment, contradicting pairwise disjointness.

A third invariant, `Typing_preserves_lc`, establishes that every well-typed process is locally closed and that every component environment in its hyperenvironment is locally closed at the current type-binder depth. The proof is split into `Typing_preserves_lc_proc` for the process and `Typing_preserves_lc_context` for the hyperenvironment, then packaged together. Both proceed by induction on the typing derivation. The co-finite binding cases use `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` to recover local closure of the closed process from that of the opened one. The `forall_` and `exists_` cases additionally require `Env.lc_shift_inv_0` and `Types.lc_subst_inv_0` to invert the effect of shifting and substitution on the environment and type indices respectively.

Substitution. `Judgement/Subst.lean` establishes that typing is stable under both forms of substitution. `Typing_substNames` states that if $n \vdash P :: \mathcal{G}$ and y does not appear in $\mathcal{G}$ except possibly as x, then substituting y for x throughout yields a well-typed process under the correspondingly substituted hyperenvironment. `Typing_substTypes` gives the analogous result for type substitution, where substituting a type A for De Bruijn index k throughout a derivation yields a well-typed process at depth $n - 1$. Both proofs proceed by induction on the typing derivation and rely on the commutation lemmas from Section 4.1.2.

4.2 Semantics

The dynamic behaviour of the πMLL fragment is formalised in the `Semantics/` namespace. Following the erasure structure described in Section 2.5, the development contains three related transition systems: a typed transition system over typing derivations, and two projected systems describing the induced process and environment behaviour.

- $\texttt{TypingStep}_{\texttt{m}}$: the primary, resource-aware LTS over typing derivations (`JudgementStep/Basic.lean`);
- $\texttt{ProcStep}_{\texttt{m}}$: the process-level LTS obtained by forgetting typing resources (`ProcStep/Basic.lean`);
- $\texttt{EnvStep}_{\texttt{m}}$: the resource-level LTS describing the corresponding evolution of hyperenvironments (`EnvStep/Basic.lean`).

Labels. Transition labels are formalised in `Labels.lean`, directly realising the label algebra introduced in Section 2.5. Although the operational semantics mechanised here target πMLL, the action vocabulary is defined for the full πLL calculus to support future extensions. The free and introduced name functions f and i are realised as `Lbl.f` and `Lbl.i`, computed by structural recursion on the label. The disjointness condition on `par` labels is enforced as the decidable predicate `Lbl.WF`, which appears as an explicit side condition in the parallel synchronisation rules.

The core action type, `Act`, captures observable communication events, including empty communication, channel transmission, type instantiation, and server interaction. Selection and server actions are parameterised by the auxiliary datatype `Mu`. The `Lbl` inductive then extends observable actions with the silent transition τ, explicit session-link labels, and a parallel constructor combining two observable actions directly, reflecting the synchronized action labels used by the parallel rules.

```
inductive Act : Type
  | one       (x : FPName)
  | tensor    (x y : FPName)
  | output    (x : FPName) (A : Types)
  | muBrack   (x : FPName) (μ : Mu)
  | ...

inductive Lbl : Type
  | tau
  | link  (x y : FPName)
  | act   (p : Act)
  | par   (l l' : Act)
```

Each label defines computable sets of free names (`Lbl.f`) and introduced names (`Lbl.i`), which are used in the side conditions for the rules $\textsc{par}_1$, $\textsc{syn}$ and $\textsc{par}_2$. To allow label notation to mirror process prefix notation, `Act` and `Lbl` implement the `HasBracket` and `HasParen` typeclasses: for instance, $x[\![y]\!]$ denotes both the output prefix and the corresponding output label.

The full definitions of the label and action inductives can be found in Section D.4.

4.2.1 The Typed Transition System. The primary LTS, $\texttt{TypingStep}_\texttt{m}$, is a relation between typing derivations. Its constructors mirror the operational rules shown in Figure 2, but make the type-theoretic content of each step explicit: every transition carries a typing derivation as its source and produces a typing derivation as its target. The system is therefore intrinsically typed.

```
inductive TypingStepₘ :
  {n : Nat} → {𝒢 : HyperEnv} → {P : Proc} → Typing n P 𝒢 →
  Lbl → {n' : Nat} → {𝒢' : HyperEnv} → {P' : Proc} → Typing n' P' 𝒢' → Prop
```

As established previously, both environment and hyperenvironment composition are formalised using total list appends. Consequently, the explicit disjointness proofs required by the static `Typing` relation to safely mix environments must propagate directly into the dynamic $\texttt{TypingStep}_\texttt{m}$ relation. For instance, the $\texttt{par}_1$ constructor requires explicit hypotheses (`hD1` and `hD2`) to guarantee that the hyperenvironments remain strictly disjoint both before and after the transition:

```
| par₁
    (hD1 : 𝒢.disjoint ℋ) (hD2 : 𝒢'.disjoint ℋ)
    (h : TypingStepₘ 𝒟 l 𝒟')
    (disj : l.i ∩ Q.f = ∅) :
    TypingStepₘ
      (Typing.mix hD1 𝒟 ℰ)
      l
      (Typing.mix hD2 𝒟' ℰ)
```

The post-state disjointness proof `hD2` is included explicitly as a convenience: while it is derivable from `hD1`, the freshness condition $i(l) \cap Q.f = \emptyset$, and the structural property that transitions only introduce names recorded by their label, keeping it as a premise avoids reconstructing this fact repeatedly in later proofs.

Remark 4.1. Strictly stepping typing derivations can expose structural artifacts that are usually hidden in the paper presentation. For example, in Figure 3, the intermediate derivation is typed by $\varnothing \mid y : 1$ rather than simply $y : 1$, because the MIX rule preserves the empty component contributed by the terminated left process. In Lean, this corresponds to a `Typing.mix` constructor containing a `Typing.mix₀` sub-derivation. The result is valid, but reducing it to the canonical form requires applying the monoidal identity laws of the hyperenvironment explicitly, rather than by definitional equality. The final $\equiv$ step in the figure, which additionally uses process congruence to identify $\mathbf{0} \mid \mathbf{0}$ with $\mathbf{0}$, is not currently automated because structural congruence has not been mechanised. This limitation is discussed further in Section 6.

4.2.2 *Projected Transition Systems.* The process and environment transition systems are defined to capture the two erasure views of a typed transition. `ProcStepₘ` records the process behaviour obtained by forgetting typing resources, while `EnvStepₘ` records the corresponding resource evolution. These relations are later connected to `TypingStepₘ` by the simulation theorems in Section 5.1.

Process steps. `ProcStepₘ` models the syntactic evolution of processes independently of typing resources. Unlike the typing relation, it is not indexed by a typing derivation and therefore does not use co-finite premises. Binder-opening rules instead choose concrete fresh names directly and record the required freshness as explicit side conditions. For example, the `tensor` rule opens the continuation with a concrete name y, requiring y to be fresh for both the subject channel and the continuation:

```
| tensor {P : Proc} {x y : FPName} (hF : y ∉ {x} ∪ P.f) :
    ProcStepₘ (#x⟦•⟧.P) (x⟦y⟧) P⦅#y⦆
```

This creates a mismatch with the co-finite formulation of the typing rules. Thus `ProcStepₘ` remains close to a standard concrete transition relation, while the additional freshness reconciliation is handled in the typability proof. A co-finite formulation of `ProcStepₘ` would align more directly with the typing rules, but the present formulation keeps the erased LTS closer to a standard concrete transition relation.

Environment steps. The structural evolution of hyperenvironments, written $\mathcal{G} \xrightarrow{l} \mathcal{H}$, is formalised by `EnvStepₘ`. Unlike the typed transition system, `EnvStepₘ` deliberately contains only the resource transformation described by the label. For example, an input action on an endpoint typed by $\perp$ consumes that resource: $\Gamma, x : \perp \xrightarrow{x()} \Gamma$. Similarly, an empty output on an endpoint typed by 1 evolves $x : 1$ to the empty environment. These rules describe how session types evolve as actions are performed, independently of any particular process term.

The relation does not independently enforce all disjointness and well-formedness conditions. Those properties are supplied by the typing derivation in `TypingStepₘ`. This separation keeps the projected resource semantics lightweight: the environment relation describes how resources change, while the typed semantics proves that these changes occur from well-formed typing environments.

The following metatheory shows that these transition systems agree: typed transitions project to process and environment transitions, and process transitions from well-typed states can be lifted back to typed transitions.

5 Metatheory

This section describes the metatheoretic results connecting the typed, process, and environment transition systems defined in Section 4.2. The development is contained in the `SessionFidelity/` namespace and focuses on session fidelity for the mechanised πMLL fragment.

5.1 Session Fidelity

Session fidelity states that well-typed processes evolve according to their session types, and that their associated linear resources evolve consistently with each process transition. The proof proceeds in two directions. First, erasure simulation shows that typed derivation steps project to valid process and environment transitions. Second, typability and subject reduction show that process transitions from well-typed processes can be lifted back to typed derivation steps and therefore preserve typing.

Erasure simulations. The paper establishes erasure as a strong bisimulation between lts_p and lts_d (Theorem 4.7 in Montesi and Peressotti [2021]), meaning the relation $\{(proc(\mathcal{D}), \mathcal{D}) \mid \mathcal{D} \in Der\}$ is closed in both directions. The mechanisation currently establishes the forward direction via `session_fidelity_procₘ`: every typed derivation step projects to a process step. The backward direction, which lifts process steps from well-typed processes to typed derivation steps, is captured by `typability_subject_reductionₘ`, but has not been packaged as an explicit bisimulation statement.

The files `ProcStep.lean` and `EnvStep.lean` establish the two projection results from typed derivation steps. Given a typed transition `TypingStepₘ` $\mathcal{D}\ l\ \mathcal{D}'$, `session_fidelity_procₘ` projects it to a process transition between $proc(\mathcal{D})$ and $proc(\mathcal{D}')$, while `session_fidelity_envₘ` projects it to an environment transition between $env(\mathcal{D})$ and $env(\mathcal{D}')$. The first result corresponds to the forward direction of the paper's erasure theorem between lts_p and lts_d. The second corresponds to derivation-level session fidelity, Theorem 4.9.

```
theorem session_fidelity_procₘ
  {𝒟 : Typing n P 𝒢} {𝒟' : Typing n' P' 𝒢'}
  (hStep : TypingStepₘ 𝒟 l 𝒟') :
  ProcStepₘ (proc 𝒟) l (proc 𝒟')
```

```
theorem session_fidelity_envₘ
  {𝒟 : Typing n P 𝒢} {𝒟' : Typing n' P' 𝒢'}
  (hStep : TypingStepₘ 𝒟 l 𝒟') :
  EnvStepₘ (env 𝒟) l (env 𝒟')
```

These results establish that typed transitions project to the corresponding process and environment transitions. Both proofs proceed by induction on the typed transition derivation. Most cases are immediate constructor translations. The main exception is formed by the `tensor` and `parr` prefix cases, where the projected process and environment rules require explicit freshness and disjointness side conditions. These are recovered from the co-finite typing premises by instantiating the continuation with a sufficiently fresh auxiliary name and using the typing invariants `Typing_f_eq_names` and `Typing_preserves_linearity`. For the process projection, this also requires relating the free names of an opened process to those of the original continuation.

These are recovered from the co-finite typing premises by instantiating the continuation with a sufficiently fresh auxiliary name and using the typing invariants `Typing_f_eq_names` and `Typing_preserves_linearity` established in **??**.

Typability. The main lifting theorem is stated in `Typability.lean`. It corresponds to the reverse direction of the erasure correspondence: given a process transition from a well-typed process, the transition can be lifted to the level of typing derivations.

This formulation differs from the paper presentation, which formulates erasure extensionally, using set-valued transition functions and equality of successor sets. In the Lean formalisation, transitions are represented as inductive relations, so the same correspondence is decomposed into relational projection and lifting directions. The theorem `session_fidelity_proc`$_m$ gives the forward projection from typed derivation steps to process steps, while `typability_subject_reduction`$_m$ gives the converse lifting direction for processes equipped with a typing derivation. This avoids defining successor sets explicitly in Lean and allows the proof to follow the induction principles generated by the transition relations.

```
theorem typability_subject_reductionₘ
  (𝒟 : Typing n P 𝒢) (hPS : ProcStepₘ P l P') :
  ∃ (𝒢' : HyperEnv) (𝒟' : Typing n P' 𝒢'),
    TypingStepₘ 𝒟 l 𝒟'
```

This theorem forms the technical core of the session fidelity argument. Its proof proceeds by induction on the process transition derivation. Each case inverts the source typing derivation to recover the typing rule compatible with the observed process shape, then constructs a target hyperenvironment and typing derivation for the resulting process.

The proof has two main technical obligations. First, it must reconcile the concrete freshness witnesses used by `ProcStep`$_m$ with the co-finite premises of the typing rules. For example, a process transition fixes a concrete name introduced by a prefix, while the corresponding typing rule provides a derivation only for all names outside some finite set. Auxiliary *all fresh* lemmas bridge this gap by showing that the co-finite premise can be instantiated with the concrete name appearing in the process step. Second, in the communication and restriction cases, the proof must rearrange hyperenvironments to expose the components containing the communicating endpoints. In particular, synchronization under restriction requires opening the restricted process with fresh endpoints, applying the induction hypothesis to the opened body, and reconstructing the target hyperenvironment up to permutation.

The remaining incomplete case is `res`. In this case, the induction hypothesis yields a typed step for the opened body of the restriction. The missing step is to invert this typed transition to recover the post-state components corresponding to the restricted endpoints, then repackage them into the co-finite premise required by the outer `cut` typing rule. This hyperenvironment reconstruction remains the main outstanding proof obligation.

Subject reduction. Subject reduction is stated in `SubjectReduction.lean`. It says that if a well-typed process takes a process transition, then the target process is well typed under some evolved hyperenvironment, and that this hyperenvironment evolves according to the same label.

```
theorem subject_reductionₘ
  {n : Nat} {𝒢 : HyperEnv} {P P' : Proc} {l : Lbl}
  (𝒟 : Typing n P 𝒢) (hPS : ProcStepₘ P l P') :
  ∃ 𝒢', EnvStepₘ 𝒢 l 𝒢' ∧ Typing n P' 𝒢'
```

The proof is a direct composition of the previous results: `typability_subject_reduction`$_m$ lifts the process step to a typed derivation step, and `session_fidelity_env`$_m$ projects that typed step to the corresponding environment transition. This realises the process-level session fidelity result corresponding to Corollary 4.10 of Montesi and Peressotti [2021].

Implementation considerations. In the mechanisation, several structural obligations that are implicit in the paper presentation become explicit. Since environment and hyperenvironment composition are implemented by total list concatenation, disjointness and freshness conditions appear as explicit premises in the typing and transition rules. Moreover, the process LTS uses concrete freshness witnesses, while the typing rules use co-finite quantification. This does not change the mathematical content of the semantics, but it introduces additional proof obligations in the typability theorem. Both issues are discussed further in Sections 6 and 7.

Interpretation. Together, these results establish the structure of the session fidelity argument for the mechanised fragment, with one remaining gap. The erasure simulations are complete, and subject reduction follows once typability is fully established. The typability theorem is proved for all multiplicative cases except `res`. In this case, the proof must show that environments not involved in the transition are preserved, up to permutation, across restriction steps. The required infrastructure is largely in place, with the remaining work concentrated in the case analysis relating uninvolved parallel environments to the post-step hyperenvironment produced by the restricted transition. Once this case is closed, the full process-level session fidelity property follows for the mechanised πMLL fragment.

The paper ([Montesi and Peressotti 2021]) additionally establishes behavioural properties including bisimilarity, the diamond property, serialisation, non-interference, and readiness for πMLL. These results are not mechanised in the current formalisation; they are discussed in Section 7.

6 Discussion

This section reflects on the design choices made throughout the formalisation, discusses alternatives that were explored, and identifies limitations of the current mechanisation.

Evolution of the session fidelity proof. The proof of session fidelity underwent significant revision before reaching its current form. An early iteration attempted to prove typability by induction on the typing derivation rather than on the process transition. This approach was poorly aligned with the theorem statement. In the parallel case, inverting a typing derivation exposes two independent sub-derivations, while the process transition determines which component actually steps. The proof therefore has no direct way to identify the sub-derivation that should be transformed and reassembled into the target typing derivation. The final proof instead proceeds by induction on $\texttt{ProcStep}_{\texttt{m}}$ and inverts the source typing derivation in each case. This aligns the induction with the observed operational step and gives direct access to the typing rule compatible with the process shape.

The treatment of environment transitions also changed during the development. An earlier approach attempted to invert $\texttt{EnvStep}_{\texttt{m}}$ directly, requiring a substantial collection of inversion lemmas, such as `EnvStep_inv_bot`, `EnvStep_inv_one`, `EnvStep_inv_one_bot`, and `EnvStep_inv_link`, together with auxiliary `HyperEnv.Perm.extract_*` lemmas for communication patterns under restriction. This infrastructure was necessary because the environment transition relation carried enough information to reconstruct hyperenvironment structure from a transition alone. The current formulation instead treats $\texttt{EnvStep}_{\texttt{m}}$ as a lightweight projection of $\texttt{TypingStep}_{\texttt{m}}$. Resource evolution is still recorded, but well-formedness and reconstruction are recovered from the typed transition. This removes a layer of environment-step inversion infrastructure and concentrates the difficult reasoning in the typability proof.

Proof-engineering tradeoffs. Several premises in the typing and transition relations are included for proof convenience rather than logical necessity. The most prominent example is the post-state disjointness proof `hD2` in the parallel constructors $\texttt{par}_1$ and $\texttt{par}_2$ of $\texttt{TypingStep}_{\texttt{m}}$, discussed in

Section 4.2.1. This premise is derivable from the pre-state disjointness proof `hD1`, the freshness condition requiring the names introduced by the label to be disjoint from the non-stepping component, and the structural property that transitions only introduce names recorded by their labels. Keeping `hD2` explicit avoids reconstructing this derived fact in every inductive case, at the cost of slightly cluttering the inductive definition. This is representative of a broader proof-engineering tradeoff: some redundancy in the definitions can make individual proofs shorter and more robust, even when it makes the relations less minimal.

A second source of proof overhead is the mismatch between concrete freshness witnesses in $\texttt{ProcStep}_{\texttt{m}}$ and co-finite in the typing rules. The binding constructors of $\texttt{ProcStep}_{\texttt{m}}$ record the particular names introduced by a transition, whereas the corresponding typing rules are stated co-finitely. The simulation proofs must therefore show that the co-finite typing premise can be instantiated with the concrete names chosen by the process transition. This is handled by auxiliary *all fresh* lemmas, such as those for the `tensor` and `parr` prefix binders. Reformulating $\texttt{ProcStep}_{\texttt{m}}$ co-finitely would align the process and typing relations more directly and could reduce the need for these freshness-bridging lemmas, although its effect on the erasure simulation proofs remains to be checked.

Structural congruence. The formalisation does not include a structural congruence relation for processes. In the paper presentation of πLL, structural congruence identifies processes modulo the commutative, associative, and unit laws of parallel composition, together with scope manipulation for restriction. The mechanisation currently recovers the typing-level counterpart of these laws through the monoidal laws for hyperenvironments and the exchange rules of the typing relation. However, it does not prove that process syntax itself is closed under structural congruence. For example, processes such as $P \mid \mathbf{0}$ and P are not automatically interchangeable; the corresponding simplification must instead be reflected manually at the level of hyperenvironments using the monoidal identity lemmas.

This omission matters because the operational semantics is defined over the concrete syntax tree of processes. In particular, synchronisation rules apply to components exposed by the syntactic structure of parallel composition. Since synchronisation labels are binary, the two processes that synchronise must be visible as sibling components in the syntax tree. For example, in a process of the form $P \mid (Q \mid R)$, a synchronisation between P and Q cannot be exposed by commutativity alone and would require associativity to rearrange the process into $(P \mid Q) \mid R$. A full structural congruence development would therefore require proving that typing and transitions are preserved under an independent congruence relation. An alternative would be to build selected symmetry and reassociation principles directly into the transition relation, for example to account for the orientation of restricted endpoints or the nesting of parallel composition. Whether this would simplify the session-fidelity proofs and later behavioural theory, or merely shift the proof burden elsewhere, remains unexplored.

Current limitations. The static formalisation covers the full πLL calculus, but the operational semantics and metatheory currently target the multiplicative fragment πMLL. Within πMLL, the remaining gap is the `res` case of $\texttt{typability\_subject\_reduction}_{\texttt{m}}$. This case requires reconstructing the post-transition hyperenvironment after stepping the opened body of a restriction. In particular, the proof must show that hyperenvironment components not involved in the transition are preserved, up to permutation, so that the outer `cut` typing derivation can be rebuilt. The required block-preservation infrastructure is largely in place, but the remaining case analysis has not yet been completed.

Several results from the paper metatheory also remain unmechanised, including the behavioural theory of bisimilarity and congruence, the diamond property, serialisation, non-interference, and readiness. These are discussed as directions for future work in the following section.

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, and freshness lemmas. Many proof steps reduce to showing that two hyperenvironments are equivalent up to permutation, that their components remain disjoint, or that a chosen name is fresh for the relevant process and environment. These obligations currently require explicit sequences of rewrites and structural lemma applications.

A dedicated `aesop` rule set equipped with the hyperenvironment permutation, disjointness, name-set, and freshness lemmas could automate much of this framing work and significantly reduce proof size. This would not replace the main inversion arguments, such as the remaining `res` case, but it could remove much of the repetitive bookkeeping around them. Similarly, aligning $\texttt{ProcStep}_\texttt{m}$ more closely with the co-finite style of the typing rules could reduce the number of explicit freshness-bridging obligations

7 Future Work

The most immediate directions for future work concern completing the remaining session-fidelity proof obligation and extending the dynamic metatheory to the full πLL calculus.

Completing the multiplicative fragment. The immediate remaining proof obligation is the `res` case of $\texttt{typability\_subject\_reduction}_\texttt{m}$. This case requires reconstructing the post-transition hyperenvironment after stepping the opened body of a restriction. The key missing ingredient is a block-preservation argument for $\texttt{TypingStep}_\texttt{m}$, showing that hyperenvironment components not involved in a transition are preserved up to permutation. This should provide the infrastructure needed to rebuild the outer `cut` typing derivation after applying the induction hypothesis to the opened body.

Co-finite quantification in ProcStep. The current $\texttt{ProcStep}_\texttt{m}$ relation uses concrete freshness witnesses for binding constructors, while the typing rules use co-finite quantification. This asymmetry introduces proof overhead particularly in the typability theorem, where the specific concrete names chosen by $\texttt{ProcStep}_\texttt{m}$ must be reconciled with the universally quantified premises of the corresponding typing rules. Refactoring $\texttt{ProcStep}_\texttt{m}$ to use co-finite quantification for its binding constructors (`tensor`, `parr`, `one_bot`, `tensor_parr` and `res`) would align the process and typing relations more closely and could reduce the need for auxiliary *all fresh* lemmas. It may also make some explicit freshness and post-state disjointness premises derivable rather than primitive, although its effect on the erasure simulation proofs remains to be explored.

Extending to full πLL. Once the multiplicative fragment is complete, extending the formalisation to the full πLL calculus appears to require additional proof engineering rather than a new formal architecture. The static model is already defined for the full calculus, and many typing inversion lemmas for the additive, polymorphic, axiom, and exponential constructs are already available. The main work is therefore expected to lie in extending $\texttt{typability\_subject\_reduction}_\texttt{m}$ and in proving the hyperenvironment extraction lemmas needed for the additional synchronisation cases, rather than in the erasure projections or the unit-like transition axioms. Some additional work would also be needed to extend the operational semantics with the transition constructors corresponding to the full set of rules from Appendix B.2 of Montesi and Peressotti [2021].

The remaining prefix and unit-like transition axioms from Appendix B.2 of Montesi and Peressotti [2021] are expected to follow the same proof pattern as the already mechanised `one` and `bot` cases. This includes the additive selections, server-use actions, polymorphic send and receive actions, disposal and duplication send actions, and the link actions. In each case, the proof should proceed by inverting the source typing derivation, constructing the corresponding typed transition, and reusing the existing local-closure and freshness infrastructure. The polymorphic cases additionally rely on the type-substitution and shifting lemmas developed for the locally nameless representation, while the link and axiom-cut cases require the name-substitution lemmas. The axiom-cut interaction may require special treatment, since it combines restriction with name substitution; one possible approach is to add an explicit axiom-under-restriction transition constructor, analogous to the existing `one_bot` and `tensor_parr` synchronisation constructors.

The remaining synchronisation cases, such as !w, !c, !?, $\exists\forall$, $\oplus_1\&$, and $\oplus_2\&$, follow the same general restriction-synchronisation pattern as the multiplicative communication cases. They require isolating the interacting components, stepping the opened body, and reconstructing the resulting hyperenvironment up to permutation. Once the extraction lemmas for the current `res` case are completed, these cases are expected to follow by analogous reasoning, with the relevant connectives and environments substituted for their multiplicative counterparts.

The duplication and disposal receive cases are more involved than the other server actions, since they generate a larger continuation process and manipulate dependencies and the typing environment. A prior revision of the formalisation, where server dependencies were stated implicitly, included the auxiliary constructions for building these processes and manipulating the corresponding dependencies and typing environments. Since the well-typedness of those constructions had already been established, parts of this earlier infrastructure may be reusable when extending the current dynamic metatheory.

Structural congruence. A further direction is to mechanise process-level structural congruence and prove that it preserves typing and transitions. This would address the artificial stuck states that arise from the strict binary-tree structure of the process syntax. In particular, commutativity and associativity of parallel composition are needed for the SYN rule to apply to processes that are not immediate siblings in the syntax tree, as discussed in Section 6. A structural congruence relation would internalise the commutative, associative, and unit laws of parallel composition, as well as scope manipulation for restriction, rather than requiring these rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Scope manipulation is especially delicate in the locally nameless setting, since moving a process under a restriction binder requires explicit De Bruijn index management. For example, a scope-extrusion principle of the form

$$(\boldsymbol{\nu}(\bullet,\bullet)\,P) \mid Q \equiv \boldsymbol{\nu}(\bullet,\bullet)\,(P \mid Q')$$

would require Q' to be obtained by shifting the bound channel indices of Q by 2, while leaving its free names unchanged, to account for the two additional channel binders introduced by the restriction. This would require both a channel-level shift operation on bound indices and a process-level traversal lifting that operation through the syntax.

The tradeoff is between modularity and local proof convenience. A full structural congruence relation would require preservation theorems showing that typing and transitions are stable under each congruence principle, but these results would then be reusable throughout the metatheory. A rule-oriented approach instead adds symmetric variants of selected rules, or selected symmetry and reassociation principles, directly to the transition relation. This could account for the congruences most relevant to stepping while preserving the syntax-directed structure of the system, which is useful for proof search in Lean. However, these additional constructors would still introduce extra cases in later proofs. Although unordered collections of processes could avoid some commutativity issues, representing process syntax quotient-wise would make syntax-directed inversion and induction substantially harder in Lean, mirroring the difficulties that led environments to be represented as lists rather than finite sets.

Behavioural metatheory. The paper establishes a range of further behavioural results that are not mechanised in the current formalisation. These include the theory of similarity, bisimilarity, and behavioural congruence, as well as the result connecting similarity with typability. The paper also proves the main parallelism properties of the calculus, namely the diamond property, serialisation, and non-interference. Finally, it establishes readiness, which for the full πLL calculus, requires the additional notion of potential, an induction metric that gives an upper bound on the number of internal transitions a process can perform. This also yields productivity: infinite executions of well-typed processes must involve infinitely many observable interactions [Montesi and Peressotti 2021].

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, name-set, and freshness lemmas. Building a dedicated `aesop` rule set equipped with these lemmas, together with a curated collection of simplification lemmas for normalising names and disjointness goals, could automate much of this framing work and significantly reduce proof size. This would however not replace the main inversion arguments, such as the remaining `res` case, but it could remove much of the repetitive bookkeeping around them. Similarly, aligning $\texttt{ProcStep}_\mathsf{m}$ more closely with the co-finite style of the typing rules could reduce the number of explicit freshness-bridging obligations.

8 Conclusion

This thesis set out to mechanise the syntax, semantics, and metatheory of πLL in Lean. The goal was not to extend the source calculus, but to validate and clarify an existing theoretical framework by making its binding structure, freshness assumptions, linear resource accounting, and operational metatheory explicit in a proof assistant.

The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, and typing judgements. Processes are represented using a locally nameless encoding, while type variables are represented using De Bruijn indices. This avoids explicit reasoning about α-equivalence while preserving readable free channel names in the mechanised syntax. The cost of this choice is the need for local-closure predicates, opening and closing operations, and co-finite reasoning principles.

The operational semantics is mechanised for the multiplicative fragment πMLL through three related transition systems: a typed transition system over typing derivations, a process transition system obtained by erasing typing resources, and an environment transition system describing the corresponding evolution of hyperenvironments. This mirrors the erasure structure of the source theory while adapting it to Lean's relational presentation of inductive transition systems.

The main metatheoretic development establishes the structure of the session-fidelity argument for the mechanised fragment. Typed derivation steps project to both process and environment transitions, and process transitions from well-typed sources can be lifted back to typed derivation steps in the completed multiplicative cases. Subject reduction is then obtained by composing this lifting result with the environment projection. The remaining proof obligation is the `res` case of $\texttt{typability\_subject\_reduction}_\mathsf{m}$, where the proof must reconstruct the post-transition hyperenvironment after stepping under restriction.

The mechanisation also exposes several proof-engineering tradeoffs. Representing environments and hyperenvironments as lists makes structural induction and inversion feasible, but requires explicit permutation, disjointness, and freshness reasoning. Similarly, the mismatch between concrete freshness witnesses in $\texttt{ProcStep}_\mathsf{m}$ and co-finite premises in the typing rules introduces auxiliary proof obligations. Finally, the absence of process-level structural congruence keeps the transition systems syntax-directed, but requires some rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Overall, the development provides a substantial mechanised account of the static structure of full πLL and the dynamic metatheory of its multiplicative fragment. It confirms that the central erasure and session-fidelity architecture of the source theory can be represented in Lean, while isolating the remaining proof infrastructure needed to complete the πMLL development and extend the mechanisation to the full calculus.

References

Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '08)*. ACM, New York, NY, USA, 3–15. doi:10.1145/1328438.1328443

Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics*, Joe Hurd and Tom Melham (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 50–65.

Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.

Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23-26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33

Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2

N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0

Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/archives/fall2023/entries/logic-linear/

Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4

Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π-calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818

Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.

A Code Availability

The complete Lean development corresponding to this thesis is submitted as a separate archive and is available at: https://github.com/MikkelBrixNielsen/LeanPiLL

The submitted archive corresponds to the version of the formalisation described in this thesis. The online repository may contain later updates, while earlier experimental versions are available in the repository history. These earlier versions are not part of the submitted artefact.

The appendix contains selected excerpts of the development. These excerpts are typeset to closely reflect the corresponding Lean code. In particular, some Unicode notation and spacing is rendered typographically rather than shown as copy-pasteable Lean source. The complete copy-pasteable Lean files, including all definitions, auxiliary lemmas, and proof scripts, are available in the accompanying codebase.

B Lean File Structure

```
PiLL/
```

```
Model/
  Environments/
  HyperEnvironments/
  Judgement/
  Processes/
  STypes/
  Base.lean
Semantics/
  EnvStep/
  JudgementStep/
  ProcStep/
  Labels.lean
SessionFidelity/
  ProcStep.lean
  EnvStep.lean
  Typability.lean
  SubjectReduction.lean
```

Listing 1. Top-level structure of the Lean development.

C Common Lean Notation

```
class HasShift (Subject : Type) where
  shift : Subject -> Nat -> Nat -> Subject

notation:max S:max " ↑ " d:max ", " c:max => HasShift.shift S d c
notation:max S:max " ↑ " c:max => HasShift.shift S 0 c
notation:max S:max "⁺" => HasShift.shift S 0 1

class HasSubst (Subject Replacement Target : Type) where
  subst : Subject -> Replacement -> Target -> Subject

notation:max S "{" R " // " T "}" => HasSubst.subst S R T

class HasOpen (Subject Replacement Target : Type) where
  open_ : Subject -> Replacement -> Target -> Subject

notation:max S:max "⦅" T " | " R "⦆" => HasOpen.open_ S R T
notation:max S:max "⦅" R "⦆" => HasOpen.open_ S R 0

class HasClose (Subject Replacement Target : Type) where
  close_ : Subject -> Replacement -> Target -> Subject

notation:max S:max "⟪" T " | " R "⟫" => HasClose.close_ S R T
notation:max S:max "⟪" R "⟫" => HasClose.close_ S R 0

class HasOpenTwo (Subject Replacement_1 Replacement_2 Target : Type) where
  open_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject

notation:max S:max "⦅" T " | " R1 ", " R2 "⦆" => HasOpenTwo.open_ S R1 R2 T
notation:max S:max "⦅" R1 ", " R2 "⦆" => HasOpenTwo.open_ S R1 R2 0
```

```
class HasCloseTwo (Subject Replacement_1 Replacement_2 Target : Type) where
  close_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject

notation:max S:max "⟪" T " | " R1 ", " R2 "⟫" => HasCloseTwo.close_ S R1 R2 T
notation:max S:max "⟪" R1 ", " R2 "⟫" => HasCloseTwo.close_ S R1 R2 0

class HasBracket (Subject Content Result : Type) where
   brack : Subject -> Content -> Result

class HasParen (Subject Content Result : Type) where
   paren : Subject -> Content -> Result

notation:80 x"⟦⟧" => HasBracket.brack x ()
notation:80 x"⟦"y"⟧" => HasBracket.brack x y
notation:80 x"⦅⦆" => HasParen.paren x ()
notation:80 x"⦅ y⦆" => HasParen.paren x y

class HasPerm (α : Type) where
  perm : α -> α -> Prop

infixr:54 " ~ " => HasPerm.perm
```

Listing 2. Common typeclasses and notation used throughout the Lean development. These abbreviations are used uniformly for shifting, substitution, opening, closing, prefix notation, and permutation.

D Selected Core Definitions

D.1 Session Types

```
abbrev Atom := Nat
abbrev FTVar := Nat
abbrev BTVar := Nat

inductive TVar : Type
  | free    (x : FTVar)
  | bound   (x : BTVar)
deriving DecidableEq, Repr

inductive Types : Type where
  | atom        (a : Atom)
  | atomDual    (a : Atom)
  | var         (v : TVar)
  | varDual     (v : TVar)
  | one
  | bot
  | tensor      (A B : Types)
  | parr        (A B : Types)
  | oplus       (A B : Types)
  | amp         (A B : Types)
  | bang        (A : Types)
  | quest       (A : Types)
  | forall_     (A : Types)
```

```
  | exists_     (A : Types)
deriving DecidableEq, BEq, Repr
```

Listing 3. Locally nameless representation of type variables and the Lean definition of session types. Free type variables are represented explicitly, while bound type variables are represented by De Bruijn indices.

D.2 Processes

This appendix gives the core Lean representation of channels and process syntax. Channels are represented using the locally nameless approach discussed in Section 3, separating free names from bound indices.

```
abbrev FPName := Nat
abbrev BPName := Nat

inductive Channel : Type where
  | free    (x : FPName)
  | bound   (x : BPName)
deriving DecidableEq, BEq, Repr

prefix:max "#" => Channel.free
prefix:max "$" => Channel.bound

inductive Proc : Type where
  | nil
  | one         (x : Channel) (P : Proc)
  | bot         (x : Channel) (P : Proc)
  | tensor      (x : Channel) (P : Proc)
  | parr        (x : Channel) (P : Proc)
  | cut         (P : Proc)
  | par         (P Q : Proc)
  | selectL     (x : Channel) (P : Proc)
  | selectR     (x : Channel) (P : Proc)
  | amp         (x : Channel) (P Q : Proc)
  | output      (x : Channel) (P : Proc) (A : Types)
  | input       (x : Channel) (P : Proc)
  | server      (x : Channel) (zs : Finset Channel) (P : Proc)
  | consume     (x : Channel) (P : Proc)
  | duplicate   (x : Channel) (P : Proc)
  | dispose     (x : Channel) (P : Proc)
  | link        (x y : Channel)
deriving DecidableEq, BEq
```

Listing 4. Locally nameless representation of channels and process syntax. Free channels are represented explicitly, while bound channels are represented by De Bruijn indices.

D.3 Judgements

```
inductive Typing : Nat -> Proc -> HyperEnv -> Prop where
  | mix∅ {n : Nat} :
      Typing n 0 ∅
```

```
| mix {𝒢 ℋ : HyperEnv} {P Q : Proc} {n : Nat} (hD : 𝒢.disjoint ℋ) :
    Typing n P 𝒢 -> Typing n Q ℋ ->
    Typing n (P |ₚ Q) (𝒢 |ₕ ℋ)

| one {P : Proc} {x : FPName} {n : Nat} :
    Typing n P ∅ ->
    Typing n (#x⟦⟧.P) [[x:1]]

| bot {Γ : Env} {P : Proc} {x : FPName} {n : Nat} (hF : x ∉ Γ.names) :
    Typing n P [Γ] ->
    Typing n (#x⦅⦆.P) [x:⊥::Γ]

| cut {𝒢 : HyperEnv} {Γ Δ : Env} {P : Proc} {A : Types} {n : Nat}
  (L : Finset FPName) :
    (∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
      Typing n (P⦅#x, #y⦆) (𝒢 |ₕ [x:A::Γ] |ₕ [y:A⊥::Δ])) ->
    Typing n (ν⦅•,•⦆ P) (𝒢 |ₕ [Γ, Δ])

| tensor {Γ Δ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat}
    (hF : x ∉ Γ.names ∧ x ∉ Δ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P⦅#y⦆) ([y:A::Γ] |ₕ [x:B::Δ])) ->
    Typing n (#x⟦•⟧.P) [x:A⊗B::Γ,Δ]

| parr {Γ : Env} {P : Proc} {x : FPName} {A B : Types}
    {n : Nat} (hF : x ∉ Γ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P⦅#y⦆) [y:A::x:B::Γ]) ->
    Typing n (#x⦅•⦆.P) [x:A⅋B::Γ]

| oplus₁ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
    B.lc n -> Typing n P [x:A::Γ] ->
    Typing n (#x⟦L⟧.P) [x:A⊕B::Γ]

| oplus₂ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
    A.lc n -> Typing n P [x:B::Γ] ->
    Typing n (#x⟦R⟧.P) [x:A⊕B::Γ]

| amp {Γ : Env} {P Q : Proc} {x : FPName} {A B : Types} {n : Nat} :
    Typing n P [x:A::Γ] -> Typing n Q [x:B::Γ] ->
    Typing n (#x.case{L : P, R : Q}) [x:A&B::Γ]

| quest {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
    Typing n P [x:A::Γ] ->
    Typing n (#x⟦USE⟧.P) [x:??A::Γ]

| bang {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
    ?ₑΓ -> Typing n P [x:A::Γ] ->
    Typing n (!#x⟨Γ.names.image Channel.free⟩.{P}) [x:!!A::Γ]

| w {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
    (hF : x ∉ Γ.names) :
    A.lc n -> Typing n P [Γ] ->
```

```
      Typing n (#x⟦DISP⟧.P) [x:??A::Γ]

  | c {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
      (hF : x ∉ Γ.names) (L : Finset FPName) :
      (∀ y ∉ L, Typing n P⦅#y⦆ [x:??A::y:??A::Γ]) ->
      Typing n (#x⟦DUP⟧⦅•⦆.P) [x:??A::Γ]

  | exists_ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
      A.lc n -> Typing n P [x:B{A // 0}::Γ] ->
      Typing n (#x⟦A⟧.P) [x:∃.B::Γ]

  | forall_ {Γ : Env} {P : Proc} {x : FPName} {B : Types} {n : Nat} :
      Typing (n + 1) P [x:B::Γ⁺] ->
      Typing n (#x⦅⋄⦆.P) [x:∀.B::Γ]

  | ax {x y : FPName} {A : Types} {n : Nat} (hneq : x ≠ y) :
      A.lc n ->
      Typing n (#x ↔ₚ #y) [x:A⊥:: [y:A]]

  | exchange_env {𝒢 : HyperEnv} {Γ Δ : Env} {P : Proc} {n : Nat} :
      Typing n P (Γ::𝒢) -> Γ ~ Δ ->
      Typing n P (Δ::𝒢)

  | exchange_hyper {𝒢 ℋ : HyperEnv} {P : Proc} {n : Nat} :
      Typing n P 𝒢 -> 𝒢 ~ ℋ ->
      Typing n P ℋ

notation:50 n " ⊢ " P " :: " 𝒢 => Typing n P 𝒢
```

Listing 5. The main typing judgement. The judgement `Typing n P G` states that process P is well typed under hyperenvironment G at type-variable depth n.

The constructors in the above listing cover the full static πLL typing relation. The multiplicative binders `cut`, `tensor`, and `parr` use co-finite premises, while `exchange_env` and `exchange_hyper` internalise exchange for environments and hyperenvironments. These exchange constructors are used throughout the metatheory to compensate for the list-based representation of contexts.

```
def proc {𝒢 : HyperEnv} {P : Proc} {n : Nat}
  (_ : n ⊢ P :: 𝒢) : Proc := P

def env {𝒢 : HyperEnv} {P : Proc} {n : Nat}
  (_ : n ⊢ P :: 𝒢) : HyperEnv := 𝒢
```

Listing 6. Projection functions from typing derivations. Since the judgement is indexed by both the process and hyperenvironment, these projections simply recover the corresponding indices.

D.4 Transition Labels

This appendix gives the full Lean definitions of the label and action datatypes discussed in Section 4.2, together with the associated free-name, introduced-name, and well-formedness functions used by the operational semantics.

```
inductive Mu : Type
  | L
  | R
  | DISP
  | DUP
  | USE
deriving Repr, DecidableEq

inductive Act : Type
  | one       (x : FPName)
  | bot       (x : FPName)
  | tensor    (x y : FPName)
  | parr      (x y : FPName)
  | output    (x : FPName) (A : Types)
  | input     (x : FPName) (A : Types)
  | muBrack   (x : FPName) (μ : Mu)
  | muParen   (x : FPName) (μ : Mu)
deriving Repr, DecidableEq

inductive Lbl : Type
  | tau
  | link  (x y : FPName)
  | act   (p : Act)
  | par   (l l' : Act)
deriving Repr, DecidableEq

@[simp] def fNamesAct : Act -> Finset FPName -- free names
  | .one x | .bot x
  | .tensor x _ | .parr x _
  | .input x _ | .output x _
  | .muBrack x _  | .muParen x _ => {x}

@[simp] def iNamesAct : Act -> Finset FPName -- introduced names
  | .one _ | .bot _ | .input _ _ | .output _ _
  | Act.muBrack _ _ | Act.muParen _ _ => ∅
  | .tensor _ y | .parr _ y => {y}

@[simp] def Lbl.f : Lbl -> Finset FPName
  | .tau        => ∅
  | .link x y   => {x, y}
  | .act a      => fNamesAct a
  | .par l l'   => fNamesAct l ∪ fNamesAct l'

@[simp] def Lbl.i : Lbl -> Finset FPName
  | .tau        => ∅
  | .link _ _   => ∅
  | .act a      => iNamesAct a
  | .par l l'   => iNamesAct l ∪ iNamesAct l'

@[simp] def Lbl.WF : Lbl -> Prop
  | .tau => True
```

```
  | .link _ _ => True
  | .act _ => True
  | .par l l' => (iNamesAct l) ∩ (iNamesAct l') = ∅
```

Listing 7. Definitions of transition labels, actions, and their associated free and introduced names.

D.5 Transition Systems

This appendix collects the full inductive definitions of the three transition relations for πMLL, as referenced from Section 4.2. The `tensor_parr` and `res` constructors of TypingStep$_\mathrm{m}$ are elided for length; their full definitions are available in the accompanying source archive.

```
inductive TypingStep_m :
    {n : Nat} -> {𝒢 : HyperEnv} -> {P : Proc} -> Typing n P 𝒢 -> Lbl ->
    {n' : Nat} -> {𝒢' : HyperEnv} -> {P' : Proc} -> Typing n' P' 𝒢' -> Prop where

  | one {P : Proc} {x : FPName} {n : Nat} {𝒟 : Typing n P ∅} :
      TypingStep_m (Typing.one (x := x) 𝒟) (x[[]]) 𝒟

  | bot {Γ : Env} {P : Proc} {x : FPName} {n : Nat}
      {hF : x ∉ Γ.names} {𝒟 : Typing n P [Γ]} :
      TypingStep_m (Typing.bot (x := x) hF 𝒟) (x(())) 𝒟

  | tensor {Γ Δ : Env} {P : Proc} {x y : FPName} {A B : Types} {n : Nat}
      {hF : x ∉ Γ.names ∧ x ∉ Δ.names} {L : Finset FPName}
      {huniq : ∀ z, z ∉ L -> Typing n (P((#z))) ([z:A::Γ] |_h [x:B::Δ])}
      {hy : y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names)} :
      TypingStep_m (Typing.tensor hF L huniq) (x[[ y]]) (Typing_tensor_all_fresh huniq y
    hy)

  | parr {Γ : Env} {P : Proc} {x y : FPName} {A B : Types} {n : Nat}
      {hF : x ∉Γ.names} {L : Finset FPName}
      {huniq : ∀ z, z ∉ L -> Typing n (P((#z))) [z :A :: x :B :: Γ]}
      {hy : y ∉({x} ∪ P.f ∪Γ.names)} :
      TypingStep_m (Typing.parr hF L huniq) (x(( y))) (Typing_parr_all_fresh huniq y hy)

  | par_1 {𝒢 ℋ 𝒢' : HyperEnv} {P Q P' : Proc} {l : Lbl} {n : Nat}
      {hD1 : 𝒢.disjoint ℋ} {hD2 : 𝒢'.disjoint ℋ}
      {𝒟 : Typing n P 𝒢} {𝒟' : Typing n P' 𝒢'} {ℰ : Typing n Q ℋ}
      (h : TypingStep_m 𝒟 l 𝒟') (disj : l.i ∩ Q.f = ∅) :
      TypingStep_m (Typing.mix hD1 𝒟 ℰ) l (Typing.mix hD2 𝒟' ℰ)

  | par_2 {𝒢 ℋ ℋ' : HyperEnv} {P Q Q' : Proc} {l : Lbl} {n : Nat}
      {hD1 : 𝒢.disjoint ℋ} {hD2 : 𝒢.disjoint ℋ'}
      {𝒟 : Typing n P 𝒢} {ℰ : Typing n Q ℋ} {ℰ' : Typing n Q' ℋ'}
      (h : TypingStep_m ℰ l ℰ') (disj : l.i ∩ P.f = ∅) :
      TypingStep_m (Typing.mix hD1 𝒟 ℰ) l (Typing.mix hD2 𝒟 ℰ')

  | syn {𝒢 𝒢' ℋ ℋ' : HyperEnv} {P P' Q Q' : Proc} {l l' : Act} {n : Nat}
      {hD1 : 𝒢.disjoint ℋ} {hD2 : 𝒢'.disjoint ℋ'}
      {𝒟 : Typing n P 𝒢} {𝒟' : Typing n P' 𝒢'}
      {ℰ : Typing n Q ℋ} {ℰ' : Typing n Q' ℋ'}
```

```
    (h_1 : TypingStep_m 𝒟 l 𝒟') (h_2 : TypingStep_m ℰ l' ℰ')
    (disj : (l |_l l').i ∩ (P |_p Q).f = ∅) (WF : (l |_l l').WF) :
    TypingStep_m (Typing.mix hD1 𝒟 ℰ) (l |_l l') (Typing.mix hD2 𝒟' ℰ')

| one_bot {𝒢 : HyperEnv} {Γ : Env} {P P' : Proc} {n : Nat} {L : Finset FPName}
    {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
      Typing n (P⦅#x, #y⦆) (𝒢 |_h [x:1::∅] |_h [y:⊥::Γ])}
    {x y : FPName}
    {hx : x ∉ ({y} ∪ P.f ∪ 𝒢.names ∪ Γ.names)}
    {hy : y ∉ (P.f ∪ 𝒢.names ∪ Γ.names)}
    {𝒟' : Typing n P' (𝒢 |_h [∅,Γ])}
    (hStep : TypingStep_m (Typing_one_bot_all_fresh huniq x y hx hy)
      (x⟦⟧ |_l y⦅⦆) 𝒟') :
    TypingStep_m (Typing.cut L huniq) (τ) 𝒟'

| tensor_parr {𝒢 : HyperEnv} {Γ Δ Ξ : Env} {P P' : Proc} {A B : Types}
    {n : Nat} {L : Finset FPName}
    {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
      Typing n (P⦅#x, #y⦆)
        (𝒢  |_h [x:A⊗B::Γ,Δ] |_h [y:A^⊥⅋B^⊥::Ξ])}
    {huniq' : ∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
              ∀ x' ∉ L, ∀ y' ∉ L, x' ≠ y' ->
              x ≠ x' -> x ≠ y' -> y ≠ x' -> y ≠ y' ->
      Typing n (P'⦅2 | #x, #y⦆⦅#x', #y'⦆)
        (𝒢 |_h [x:B::Δ] |_h [x':A::Γ] |_h [y':A^⊥::y:B^⊥::Ξ])}
    {x y : FPName} (hx : x ∉ L) (hy : y ∉ L) (hneq : x ≠ y)
    {x' y' : FPName} (hx' : x' ∉ L) (hy' : y' ∉ L) (hneq' : x' ≠ y')
    (hxx' : x ≠ x') (hxy' : x ≠ y') (hyx' : y ≠ x') (hyy' : y ≠ y')
    (hxP : x ∉ P.f) (hyP : y ∉ P.f) (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
    (hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f) (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
    (hStep : TypingStep_m (huniq x hx y hy hneq)
      (x⟦x'⟧ |_l y⦅y'⦆)
      (huniq' x hx y hy hneq x' hx' y' hy' hneq' hxx' hxy' hyx' hyy')) :
    TypingStep_m (Typing.cut L huniq) (τ)
      (Typing_double_cut_tensor_parr huniq')

| res {𝒢 𝒢' : HyperEnv} {Γ Γ' Δ Δ' : Env} {P P' : Proc} {A : Types}
    {n : Nat} {l : Lbl} {L L' : Finset FPName}
    {huniq : ∀ z ∉ L, ∀ w ∉ L, z ≠ w ->
      Typing n (P⦅#z, #w⦆) (𝒢 |_h [z:A::Γ] |_h [w:A^⊥::Δ])}
    {huniq' : ∀ z ∉ L', ∀ w ∉ L', z ≠ w ->
      Typing n (P'⦅#z, #w⦆) (𝒢' |_h [z:A::Γ'] |_h [w:A^⊥ :: Δ'])}
    {x y : FPName} (hneq : x ≠ y)
    (hx_pre : x ∉ P.f ∪ 𝒢.names ∪ Γ.names ∪ Δ.names)
    (hy_pre : y ∉ P.f ∪ 𝒢.names ∪ Γ.names ∪ Δ.names)
    (hx_post : x ∉ P'.f ∪ 𝒢'.names ∪ Γ'.names ∪ Δ'.names)
    (hy_post : y ∉ P'.f ∪ 𝒢'.names ∪ Γ'.names ∪ Δ'.names)
    (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i)
    (hStep : TypingStep_m
      (Typing_res_all_fresh huniq x y hx_pre hy_pre hneq) l
      (Typing_res_all_fresh huniq' x y hx_post hy_post hneq)) :
    TypingStep_m (Typing.cut L huniq) l (Typing.cut L' huniq')
```

```
  | perm_env {𝒢 ℋ : HyperEnv} {Γ Γ′ : Env} {P P' : Proc} {n n' : Nat} {l : Lbl}
      {𝒟 : n ⊢ P :: Γ::𝒢} {𝒟' : n ⊢ P' :: ℋ} (hP1 : Γ ~ Γ′)
      (hTS : TypingStep_m 𝒟 l 𝒟') :
      TypingStep_m (Typing.exchange_env 𝒟 hP1) l 𝒟'

  | perm_hyper {𝒢 𝒢′ ℋ ℋ′ : HyperEnv} {P P' : Proc} {n n' : Nat} {l : Lbl}
      {𝒟 : n ⊢ P ⊢ 𝒢} {𝒟' : n' ⊢ P' ⊢ ℋ}
      (hP1 : 𝒢 ~ 𝒢′) (hP2 : ℋ ~ ℋ′)
      (hTS : TypingStep_m 𝒟 l 𝒟') :
      TypingStep_m (Typing.exchange_hyper 𝒟 hP1) l
        (Typing.exchange_hyper 𝒟' hP2)
```

Listing 8. The typed transition relation $\mathrm{TypingStep}_m$.

```
inductive EnvStep_m : HyperEnv -> Lbl -> HyperEnv -> Prop where
  | one {x : FPName} :
      EnvStep_m [[x : 1]] (x[[]]) ∅
  | tensor {Γ Δ : Env} {x x' : FPName} {A B : Types}
      (hF : x' ∉HyperEnv.names [x : A ⊗ B :: Γ Δ]) :
      EnvStep_m [x : A ⊗ B :: Γ Δ] (x[[ x']]) ([x' : A :: Γ] |_h [x : B :: Δ])
  | bot {Γ : Env} {x : FPName} :
      EnvStep_m [x : ⊥ :: Γ] (x(())) [Γ]
  | parr {Γ : Env} {x x' : FPName} {A B : Types}
      (hF : x' ∉HyperEnv.names [x : A ⅋ B :: Γ]) :
      EnvStep_m [x : A ⅋ B :: Γ] (x(( x'))) [x' : A :: x : B :: Γ]
  | par₁ {𝒢 𝒢′ ℋ : HyperEnv} {l : Lbl} :
      EnvStep_m 𝒢 l 𝒢′ ->
      EnvStep_m (𝒢 |_h ℋ) l (𝒢′ |_h ℋ)
  | par₂ {𝒢 ℋ ℋ′ : HyperEnv} {l : Lbl} :
      EnvStep_m ℋ l ℋ′ ->
      EnvStep_m (𝒢 |_h ℋ) l (𝒢 |_h ℋ′)
  | syn {𝒢 𝒢′ ℋ ℋ′ : HyperEnv} {l l' : Act} :
      EnvStep_m 𝒢 l 𝒢′ -> EnvStep_m ℋ l' ℋ′ ->
      EnvStep_m (𝒢 |_h ℋ) (l |_1 l') (𝒢′ |_h ℋ′)
  | one_bot {𝒢 : HyperEnv} {Γ : Env} {x y : FPName} :
      EnvStep_m (𝒢 |_h [[x : 1]] |_h [y : ⊥ :: Γ]) (x[[]] |_1 y(())) (𝒢 |_h [Γ]) ->
      EnvStep_m (𝒢 |_h [Γ]) (τ) (𝒢 |_h [Γ])
  | tensor_parr {𝒢 : HyperEnv} {Γ Δ Ξ : Env} {x x' y y' : FPName} {A B : Types} :
      EnvStep_m
        (𝒢 |_h [x : A ⊗ B :: Γ Δ] |_h [y : A^⊥ ⅋ B^⊥ :: Ξ])
        (x[[ x']] |_1 y(( y')))
        (𝒢 |_h [x : B :: Δ] |_h [x' : A :: Γ] |_h [y' : A^⊥ :: y : B^⊥ :: Ξ]) ->
      EnvStep_m (𝒢 |_h [Γ Δ Ξ]) (τ) (𝒢 |_h [Γ Δ Ξ])
  | res {𝒢 𝒢′ : HyperEnv} {Γ Γ′ Δ Δ′ : Env}
      {x y : FPName} {A : Types} {l : Lbl}
      (hFx : x ∉ l.i ∪ l.f) (hFy : y ∉ l.i ∪ l.f) :
      EnvStep_m
        (𝒢 |_h [x : A :: Γ] |_h [y : A^⊥ :: Δ]) l
        (𝒢′ |_h [x : A :: Γ′] |_h [y : A^⊥ :: Δ′]) ->
      EnvStep_m (𝒢 |_h [Γ Δ]) l (𝒢′ |_h [Γ′ Δ′])
  | perm_env {𝒢 ℋ : HyperEnv} {Γ Γ′ : Env} {l : Lbl}
```

```
    (hP : Γ ~ Γ') :
    EnvStep_m (Γ :: 𝒢) l ℋ ->
    EnvStep_m (Γ' :: 𝒢) l ℋ
  | perm_hyper {𝒢 𝒢' ℋ ℋ' : HyperEnv} {l : Lbl}
    (hP1 : 𝒢 ~ 𝒢') (hP2 : ℋ ~ ℋ') :
    EnvStep_m 𝒢 l ℋ ->
    EnvStep_m 𝒢' l ℋ'
```

Listing 9. The environment-level transition relation $\mathsf{EnvStep}_m$.

```
inductive ProcStep_m : (P : Proc) -> Lbl -> (P' : Proc) -> Prop where
  | one {P : Proc} {x : FPName} :
    ProcStep_m (#x⟦⟧· P) (x⟦⟧) P
  | tensor {P : Proc} {x y : FPName} (hF : y ∉{x} ∪ P.f) :
    ProcStep_m (#x⟦•⟧· P) (x⟦ y⟧) P⦅#y⦆
  | bot {P : Proc} {x : FPName} :
    ProcStep_m (#x⦅⦆· P) (x⦅⦆) P
  | parr {P : Proc} {x y : FPName} (hF : y ∉{x} ∪ P.f) :
    ProcStep_m (#x⦅•⦆· P) (x⦅ y⦆) P⦅#y⦆
  | par₁ {P P' Q : Proc} {l : Lbl} :
    ProcStep_m P l P' -> l.i ∩Q.f = ∅->
    ProcStep_m (P |_p Q) l (P' |_p Q)
  | par₂ {P Q Q' : Proc} {l : Lbl} :
    ProcStep_m Q l Q' -> l.i ∩ P.f = ∅->
    ProcStep_m (P |_p Q) l (P |_p Q')
  | syn {P P' Q Q' : Proc} {l l' : Act} :
    ProcStep_m P l P' -> ProcStep_m Q l' Q' ->
    (l |_l l').i ∩(P |_p Q).f = ∅-> (l |_l l').WF ->
    ProcStep_m (P |_p Q) (l |_l l') (P' |_p Q')
  | one_bot {P P' : Proc} {x y : FPName}
    (hx : x ∉ P.f) (hy : y ∉ P.f) (hneq : x ≠ y) :
    ProcStep_m P⦅#x, #y⦆ (x⟦⟧ |_l y⦅⦆) P' ->
    ProcStep_m (ν⦅•,•⦆ P) (τ) P'
  | tensor_parr {P P' : Proc} {x x' y y' : FPName}
    (hxP : x ∉ P.f) (hyP : y ∉ P.f)
    (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
    (hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f)
    (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
    (hxx' : x ≠ x') (hxy : x ≠ y) (hxy' : x ≠ y')
    (hyx' : y ≠ x') (hyy' : y ≠ y') (hx'y' : x' ≠ y') :
    ProcStep_m P⦅#x, #y⦆ (x⟦ x'⟧ |_l y⦅ y'⦆) P'⦅2 | #x, #y⦆⦅#x', #y'⦆ ->
    ProcStep_m (ν⦅•,•⦆ P) (τ) (ν⦅•,•⦆ (ν⦅•,•⦆ P'))
  | res {P P' : Proc} {l : Lbl} {x y : FPName}
    (hneq : x ≠ y)
    (hx : x ∉ P.f ∪ P'.f) (hy : y ∉ P.f ∪ P'.f)
    (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i) :
    ProcStep_m P⦅#x, #y⦆ l P'⦅#x, #y⦆ ->
    ProcStep_m (ν⦅•,•⦆ P) l (ν⦅•,•⦆ P')
```

Listing 10. The process-level transition relation $\mathsf{ProcStep}_m$.

E Erasure Functions and Main Theorems

These things should be included in this appendix

```
- session_fidelity_proc
- session_fidelity_env
- typability_subject_reduction
- subject_reduction
```

F Selected Proof Infrastructure

These things should be indcluded in this appendix

```
Freshness/co-finite bridge:
- Typing_tensor_all_fresh
- Typing_parr_all_fresh
- Typing_res_all_fresh if is discussed, else something similar

Typing inversion:
- Typing_inv_one
- Typing_inv_tensor
- maybe Typing_inv_cut

Hyperenvironment extraction/block preservation:
- one representative HyperEnv.Perm.extract_* statement
- optionally one block-preservation lemma statement if it is clean enough TypingStep_preser
```